

JSPM's
RAJARSHI SHAHU COLLEGE OF ENGINEERING
TATHAWADE, PUNE-33
(An Autonomous Institute Affiliated to Savitribai Phule Pune University,Pune)



LAB MANUAL

OF

Parallel and Distributed Computing Lab

(IT4201)

B.Tech

INFORMATION TECHNOLOGY

Authors: 1) Dr. Sandeep Chaware
2) Prof. Dewendra Bharambe

Creation Date: - June 2026

Last Updated: - July 2026

Version: - 2

RSCOE-IT

INDEX			
Sr. No.		Topic	Page. No.
1		How to Use This Manual…….	3

RSCOE-IT

2		**Laboratory Details……**	
	2.1	**Laboratory Objective**	**8**
	2.2	**Laboratory Equipment**	**9**
	2.4	**Experiment No.1**	**10**
		Aim, theory, procedure, observations, calculations etc	
	2.5	**Experiment No.2.**	**13**
		Aim, theory, procedure, observations, calculations etc	
	2.6	**Experiment No.3**	**19**
		Aim, theory, procedure, observations, calculations etc	
	2.7	**Experiment No.4**	**32**
		Aim, theory, procedure, observations, calculations etc	
	2.8	**Experiment No.5**	**38**
		Aim, theory, procedure, observations, calculations etc	
	2.9	**Experiment No.6**	**48**
		Aim, theory, procedure, observations, calculations etc	

© JSPM Group of Institutes, Pune. All Rights Reserved. All the information in this Course Manual is confidential. Participants shall refrain from copying, distributing, misusing/ or disclosing the content to any third parties any circumstances whatsoever.

1. How to Use This Manual

This manual assumes that the facilitators are aware of collaborative learning methodologies.
This manual will provide a tool to facilitate the session on Digital Communication modules in collaborative learning environment.
The facilitator is expected to refer this manual before the session.

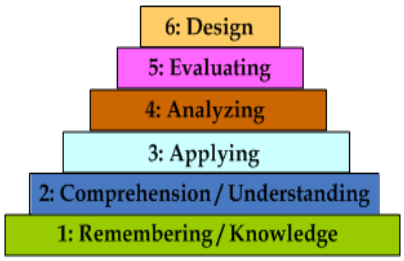
IconofGraduateAttributes

K Applying Knowledge	**A** Problem Analysis	**D** Design & Development	**I** Investigation of problems
M Modern Tool Usage	**E** Engineer & Society	**E** Environment Sustainability	**T** Ethics
T Individual & Team work	**O** Communication	**M** Project Management & Finance	**I** Life Long Learning

Disk Approach- Digital Blooms Taxonomy



1: Remembering / Knowledge
2: Comprehension / Understanding
3: Applying
4: Analyzing
5: Evaluating
6: Creating / Design



This Manual uses icons as visual cues to the interactivities during the session.

	Blooms Taxonomy
	Remembering
	Understanding

	Applying
	Analyzing
	Evaluating
	Creating

	This icon is used to indicate instructions for faculties.
	This icon is used to indicate a statement to be made by faculty.
	This icon is used to indicate a list of additional resources.
	This icon indicates an activity to be conducted.
	This icon indicates questions to be asked by faculty.

RSCOE,IT

VISION of Institute
To be a recognized institution for academic excellence, innovation, research, leadership and holistic development for socially responsible professionals driving sustainable national and global progress.

MISSION of Institute
M1: To deliver transdisciplinary, outcome-based, and experiential education that nurtures academic excellence, critical thinking, professional competence, and lifelong learning.

M2: To promote research, innovation, entrepreneurship, and collaborative knowledge creation to address national and global challenges.

M3: To instil ethical values, leadership, social responsibility, and global competence for holistic and sustainable human development.

VISION of Department
To achieve excellence in education, research, and innovation by developing ethical and globally competent IT professionals contributing to sustainable technological progress and societal well-being.

MISSION of Department
M1. To impart quality IT education through an industry-relevant curriculum and outcome- based experiential learning to nurture competent lifelong learners.

M2 : To strengthen research, innovation, and entrepreneurship in emerging information technology through industry collaboration to develop technology-driven solutions.

M3. To promote the holistic development of ethical IT professionals through leadership, social commitment, and a global perspective for sustainable development.

Program Educational Objectives

PEO1: Graduates shall establish successful careers in Information Technology and allied domains by applying professional competence, lifelong learning skills, ethical values, and leadership qualities.

PEO2: Graduates shall pursue higher studies, research, innovation, and entrepreneurship in emerging areas of information technology to address industry needs and societal challenges responsibly.

PEO3: Graduates shall contribute to sustainable development and societal welfare through technology-driven solutions while demonstrating global competence, professional ethics, and continuous learning.

Program Specific Outcomes

PSO1: Integrate principles and fundamentals of computing, software engineering, data management, networking and programming knowledge to analyze, design, and develop efficient technology solutions for real-world problems.

RSCOE-IT

PSO2: Apply emerging technologies such as artificial intelligence, machine learning, cyber security, cloud computing, data analytics, digital forensics and multidisciplinary skills to develop innovative, secure, and sustainable solutions for societal and industrial needs.

PSO3: Develop professional competence, leadership, ethical values and entrepreneurial skills to engage in lifelong learning and contribute to national and global technological advancements.

PROGRAM OUTCOMES (POs)

Engineering Graduates will be able to:

PO1: Engineering Knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

PO2. Problem Analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

PO3. Design/Development of Solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

PO4. Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

PO5. Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

PO6. The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

PO7. Environment and sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

PO8. Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

PO9. Individual and team work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

PO10. Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

PO11. Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PO12. Life-long learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

RSOE-IT

Course Prerequisites:

Web Technology , Computer Network Technology , Operating System

Course Objectives:

- Understand theories and practices in parallel computing
- To understand the fundamentals and knowledge of the architectures of distributed systems.
- To gain knowledge of working components and fault tolerance of distributed systems
- To make students aware about security issues and protection mechanism for distributed environment.

Course Outcomes: After successful completion of the course, students will able to

- **CO1**: Describe parallel programming & distributed computing concepts.
- **CO2**: Implement parallel algorithm using OpenMP.
- **CO3**: Apply parallel & distributed computing techniques to solve real world problems.
- **CO4**: Design and develop CUDA-based parallel applications.

Laboratory Objectives

- Study Parallel Architectures and algorithms for Parallel computing
- Learn Distributed system architectures and communication mechanism for distributed computing
- Acquire knowledge of CUDA programming

Laboratory Outcomes:

- **LO1** : Write programs using OpenMP environment
- **LO2** : Implement communication mechanism for distributed system using RMI, CORBA etc.
- **LO3** : Use CUDA toolkit for parallel programing

Upon completion of student should able Parallel and Distributed Computing to:

1. Study Basics of OpenMP API (Open Multi-Processor API) and write program for addition of 8 numbers using threads
2. Implement any two searching and sorting algorithm on parallel systems using multi-threading
3. To develop any distributed application using Message Passing Interface (MPI)
4. To develop any distributed application through implementing client-server communication programs

RSCOE-IT

based on Java Sockets and RMI techniques

5 To develop any distributed application using Messaging System in Publish-Subscribe paradigm.

6. To create a simple web service and write any distributed application to consume the web service

7. To develop any distributed application with CORBA program using JAVA IDL

8. To use CUDA toolkit on windows / Linux OS to write programs

Laboratory Equipment list

Exp. No.	**Title of Experiment**	**Session**
1.	Study Basics of OpenMP API (Open Multi-Processor API) and write program for addition of 8 numbers using threads	
2.	Implement any two searching and sorting algorithm on parallel systems using multi-threading	
3.	To develop any distributed application using Message Passing Interface (MPI)	
4.	To develop any distributed application through implementing client-server communication programs based on Java Sockets and RMI techniques	
5.	To develop any distributed application using Messaging System in Publish-Subscribe paradigm	
6.	To create a simple web service and write any distributed application to consume the web service	
7.	To develop any distributed application with CORBA program using JAVA IDL	
8.	To use CUDA toolkit on windows / Linux OS to write programs	
	End semester assessment	
	Mock Practical Examination	

RSCOE-IT

Experiment 1

Study Basics of OpenMP API (Open Multi-Processing API) and Write a Program for Addition of 8 Numbers Using Threads

1. Aim

To study the basics of the **OpenMP API (Open Multi-Processing API)** and implement a parallel program for the addition of **8 numbers using multiple threads**.

2. Course Outcomes (COs) Addressed

CO No.	Course Outcome
CO1	Apply OpenMP constructs to develop parallel programs using shared memory architecture.
CO2	Analyze the performance improvement achieved using parallel programming techniques.

3. Program Outcomes (POs) Mapped

PO	Description	Mapping Level
PO1	Engineering Knowledge	3
PO2	Problem Analysis	2
PO3	Design/Development of Solutions	2
PO5	Modern Tool Usage	3

4. Learning Objectives

After completing this experiment, students will be able to:

- Understand the concept of shared-memory parallel programming.
- Study the OpenMP programming model.
- Learn OpenMP directives and runtime library routines.
- Create multiple threads using OpenMP.
- Implement a parallel program for summation using threads.
- Compare sequential and parallel execution.

5. Theory

5.1 Introduction to OpenMP

OpenMP (**Open Multi-Processing**) is an **Application Programming Interface (API)** designed for **shared-memory parallel programming** in **C, C++ and Fortran**. It enables programmers to parallelize existing sequential programs by using compiler directives without significantly changing the program structure.

OpenMP uses the **Fork-Join Execution Model**, where a master thread creates multiple worker threads to execute a parallel region. After execution, all worker threads synchronize and join back with the master thread.

OpenMP consists of three major components:

- Compiler Directives
- Runtime Library Functions
- Environment Variables

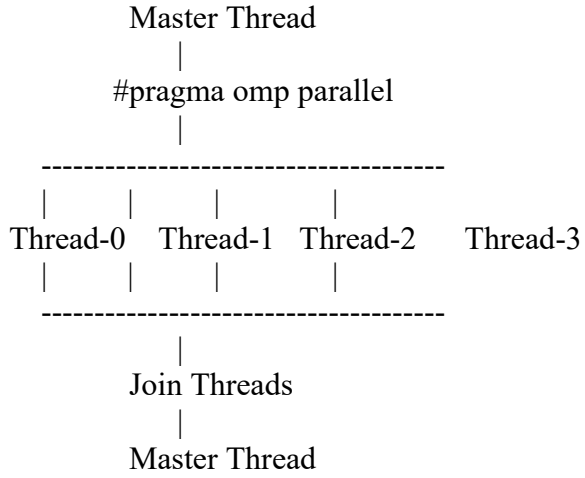
5.2 Features of OpenMP

- Portable API
- Shared Memory Programming Model
- Easy to Learn
- Automatic Thread Management

RSCOE-IT

- Incremental Parallelization
- Efficient Loop Parallelization
- Synchronization Support
- Reduction Operations
- Runtime Scheduling
- Scalability on Multi-core Systems

5.3 OpenMP Execution Model



5.4 OpenMP Programming Model

OpenMP supports

- Parallel Regions
- Work Sharing
- Synchronization
- Thread Management
- Reduction Operations
- Scheduling Policies

6. OpenMP Directives

Directive	Description
#pragma omp parallel	Creates multiple threads
#pragma omp parallel for	Executes loop iterations in parallel
#pragma omp critical	Allows only one thread at a time
#pragma omp barrier	Synchronizes all threads
#pragma omp atomic	Performs atomic update
#pragma omp master	Executes block only by master thread
#pragma omp single	Executes block by only one thread
reduction	Combines partial results

7. OpenMP Runtime Library Functions

Function	Purpose
omp_get_thread_num()	Returns current thread ID
omp_get_num_threads()	Returns total number of threads
omp_get_max_threads()	Returns maximum available threads
omp_set_num_threads()	Sets the number of threads
omp_get_wtime()	Returns execution time

KCOE-IT

8. Software and Hardware Requirements

Hardware

- Desktop/Laptop
- Intel i3/i5/i7 or AMD Processor
- Minimum 4 GB RAM

Software

- Windows/Linux Operating System
- GCC Compiler (with OpenMP Support)
- Visual Studio Code / Code::Blocks / Dev-C++
- OpenMP Library

9. Algorithm

1. Start the program.
2. Declare an array containing eight numbers.
3. Initialize the variable sum = 0.
4. Set the required number of OpenMP threads.
5. Create a parallel region using the parallel for directive.
6. Divide loop iterations among available threads.
7. Each thread computes its partial sum.
8. Use the reduction clause to combine partial sums.
9. Display the final sum.
10. Stop the program.

10. Program

```
#include <stdio.h>
#include <omp.h>

int main()
{
   int arr[8] = {10,20,30,40,50,60,70,80};
   int sum = 0;

   omp_set_num_threads(4);

   #pragma omp parallel for reduction(+:sum)
   for(int i = 0; i < 8; i++)
   {
      printf("Thread %d is adding %d\n",
            omp_get_thread_num(), arr[i]);

      sum += arr[i];
   }

   printf("\nTotal Sum = %d\n", sum);

   return 0;
}
```

RSCOE-IT

11. Program Explanation

Step 1

#include <stdio.h>
#include <omp.h>
Imports the Standard Input/Output library and the OpenMP library.

Step 2

int arr[8]={10,20,30,40,50,60,70,80};
Stores the eight integers to be added.

Step 3

omp_set_num_threads(4);
Creates four worker threads.

Step 4

#pragma omp parallel for reduction(+:sum)

- Creates multiple threads.
- Divides loop iterations among them.
- Each thread computes a local sum.
- The reduction clause safely combines all local sums into the final result.

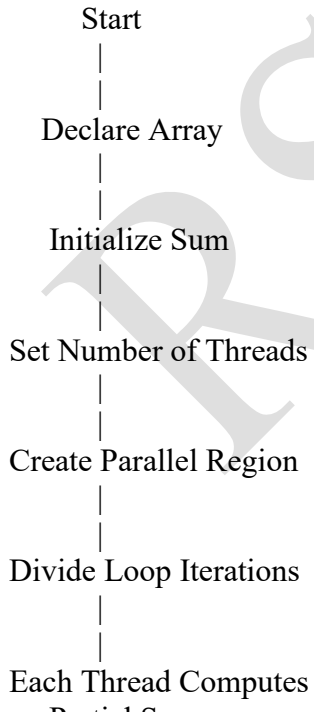
Step 5

omp_get_thread_num()
Returns the unique ID of the thread executing the current iteration.

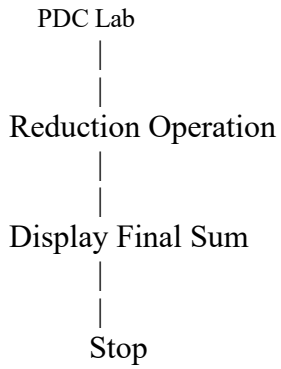
Step 6

printf("Total Sum=%d",sum);
Displays the final sum after all threads complete execution.

12. Flowchart



RSCOE-IT



13. Sample Output

Thread 0 is adding 10
Thread 0 is adding 20
Thread 1 is adding 30
Thread 1 is adding 40
Thread 2 is adding 50
Thread 2 is adding 60
Thread 3 is adding 70
Thread 3 is adding 80

Total Sum = 360

Note: The order of thread execution may vary on different executions.

14. Dry Run

Thread	**Elements Processed**	**Partial Sum**
Thread 0	10, 20	30
Thread 1	30, 40	70
Thread 2	50, 60	110
Thread 3	70, 80	150
Final Sum	**All Elements**	**360**

15. Result

The OpenMP program was executed successfully, and the sum of **8 numbers** was computed correctly using multiple threads. The experiment demonstrated how OpenMP distributes work among threads and combines partial results using the **reduction** clause.

RSCOE-IT

Experiment No. 2

1. Aim

To implement **parallel searching** and **parallel sorting** algorithms using **OpenMP multi-threading**, and compare their execution with sequential algorithms.

2. Course Outcomes (COs) Addressed

CO No.	Course Outcome
CO1	Apply OpenMP constructs to develop parallel programs.
CO2	Implement parallel algorithms using multiple threads.
CO5	Analyze scalability and performance of parallel programs.

3. Program Outcomes (POs) Mapped

PO	Description	Level
PO1	Engineering Knowledge	3
PO2	Problem Analysis	2
PO3	Design Solutions	2
PO5	Modern Tool Usage	3

4. Learning Objectives

After completing this experiment, students will be able to:

- Understand parallel searching algorithms.
- Understand parallel sorting algorithms.
- Develop OpenMP programs using multiple threads.
- Apply loop parallelization.
- Compare sequential and parallel execution.
- Analyze performance improvement using parallel computing.

5. Theory

5.1 Introduction

Searching and sorting are two fundamental operations used in almost every computing application.
When datasets become very large, sequential algorithms require considerable execution time. Parallel computing distributes work among multiple processors or threads, thereby improving performance.
Using **OpenMP**, searching and sorting operations can execute simultaneously on different portions of the data.

6. Parallel Searching

Definition

Parallel searching divides the data among multiple threads.
Each thread searches its assigned portion independently.
If any thread finds the required element, it reports the position.

Example

Array
10 20 30 40 50 60 70 80
Target = **70**
Using four threads

Thread	Elements
T0	10 20

RSCOE-IT

T1	30 40
T2	50 60
T3	70 80

Thread 3 finds the element.

Advantages

- Faster searching
- Better CPU utilization
- Efficient for large datasets
- Scalable

7. Parallel Sorting

Definition

Parallel sorting divides the array into smaller portions.
Each thread sorts one portion independently.
Finally, sorted portions are merged into a single sorted array.

Example

Original Array
70 10 80 20 60 30 50 40
Thread 1
70 10 80 20
↓
Sorted
10 20 70 80
Thread 2
60 30 50 40
↓
Sorted
30 40 50 60
↓
Merged
10 20 30 40 50 60 70 80

8. Algorithms Selected

This experiment implements:

Searching Algorithm

- Parallel Linear Search

Sorting Algorithm

- Parallel Bubble Sort (Odd-Even Transposition Sort)

RSCOE-IT

PART-A
Program 1 – Parallel Linear Search Using OpenMP

Algorithm

1. Start.
2. Read array elements.
3. Read the key element.
4. Set the number of OpenMP threads.
5. Divide the array among threads.
6. Each thread searches its assigned elements.
7. If the key is found, store its index.
8. Display the search result.
9. Stop.

Program

```
#include<stdio.h>
#include<omp.h>

int main()
{
   int arr[8]={10,20,30,40,50,60,70,80};
   int key=70;
   int found=-1;

   omp_set_num_threads(4);

   #pragma omp parallel for
   for(int i=0;i<8;i++)
   {
      if(arr[i]==key)
      {
         #pragma omp critical
         {
            found=i;
            printf("Thread %d found %d at index %d\n",
                  omp_get_thread_num(),key,i);
         }
      }
   }

   if(found!=-1)
      printf("\nElement Found at Index %d\n",found);
   else
      printf("\nElement Not Found\n");

   return 0;
}
```

RSCOE-IT

Sample Output

Thread 3 found 70 at index 6

Element Found at Index 6

Program Explanation

- Four threads are created.
- Each thread searches a portion of the array.
- When the element is found, the critical section safely updates the shared variable found.
- The program reports the index of the searched element.

PART-B

Program 2 – Parallel Bubble Sort (Odd-Even Transposition Sort)

Algorithm

1. Start.
2. Read the array.
3. Create OpenMP threads.
4. Perform odd-even comparison phases.
5. Swap adjacent elements where necessary.
6. Repeat until the array is sorted.
7. Display the sorted array.
8. Stop.

Program

```
#include<stdio.h>
#include<omp.h>

int main()
{
   int arr[8]={70,10,80,20,60,30,50,40};

   int n=8;

   omp_set_num_threads(4);

   for(int phase=0;phase<n;phase++)
   {
      if(phase%2==0)
      {
         #pragma omp parallel for
         for(int i=1;i<n;i+=2)
         {
            if(arr[i-1]>arr[i])
            {
               int temp=arr[i];
               arr[i]=arr[i-1];
               arr[i-1]=temp;
            }
```

RSCOE-IT

```
        }
      }
      else
      {
        #pragma omp parallel for
        for(int i=1;i<n-1;i+=2)
        {
          if(arr[i]>arr[i+1])
          {
            int temp=arr[i];
            arr[i]=arr[i+1];
            arr[i+1]=temp;
          }
        }
      }
    }

    printf("Sorted Array:\n");

    for(int i=0;i<n;i++)
      printf("%d ",arr[i]);

    return 0;
}
```

Sample Output

Sorted Array

10 20 30 40 50 60 70 80

Program Explanation

- The algorithm performs multiple odd and even comparison phases.
- Independent comparisons within each phase execute in parallel.
- After all phases, the array becomes fully sorted.

9. Flowchart

Start
|
Read Input Array
|
Create Threads
|
Parallel Searching/
Parallel Sorting
|
Collect Results
|
Display Output
|

RSCOE-IT

Stop

10. Execution Steps in Ubuntu Linux
Step 1
Open Terminal
Ctrl + Alt + T

Step 2
Create Program
nano search.c
or
nano sort.c

Step 3
Compile
gcc -fopenmp search.c -o search
gcc -fopenmp sort.c -o sort

Step 4
Execute
./search
./sort

11. Expected Output
Search
Thread 3 found 70 at index 6

Element Found at Index 6

Sort
Sorted Array

10 20 30 40 50 60 70 80

12. Time Complexity

Algorithm	Sequential	Parallel (Approx.)
Linear Search	O(n)	O(n/p)
Odd-Even Bubble Sort	$O(n^2)$	$O(n^2/p)$ (phase-wise parallelism)

Where **n** is the number of elements and **p** is the number of threads.

13. Applications
- Database Management Systems
- Search Engines
- Artificial Intelligence
- Machine Learning
- Image Processing
- Scientific Computing
- Data Analytics

RSCOE-IT

- Cloud Computing
- High Performance Computing
- Operating Systems

14. Advantages

- Reduced execution time
- Better processor utilization
- Improved scalability
- Increased throughput
- Suitable for multi-core processors

15. Limitations

- Thread creation overhead
- Synchronization overhead
- Communication cost
- Load imbalance for irregular data
- Performance gains are significant mainly for large datasets

16. Result

The implementation of **Parallel Linear Search** and **Parallel Bubble Sort (Odd-Even Transposition Sort)** using OpenMP was executed successfully. The experiment demonstrated how multiple threads can work concurrently to improve the performance of searching and sorting operations on shared-memory systems.

RSCOE-IT

EXPERIMENT NO.3

To develop any distributed application using Message Passing Interface (MPI).

Title: To develop any distributed application using Message PassingInterface (MPI)

Objectives: To Design a distributed application using Message Passing Interface (MPI) for remote computationwhere client submits a string to the server and server returns the reverse of it to the client

Aim/ Problem Statement:: The scope of the assignment is to cover the following issues:

1. Client-Server communication implementation.
2. Request will be send between Client to Server for getting services from Server

Theory:

MPI (Message-Passing Interface) is a message-passing library interface specification. All parts of this definition are significant. MPI addresses primarily the message-passing parallel programming model, in which data is moved from the address space of one process to that of another process through cooperative operations on each process. Extensions to the \classical" message-passing model are provided in collective operations, remote-memory access operations, dynamic process creation, and parallel I/O. MPI is a significant, not an implementation; there are multiple implementations of MPI. This significant is for a library interface; MPI is not a language, and all MPI operations are expressed as functions, subroutines, or methods, according to the appropriate language bindings which, for C and Fortran, are part of the MPI standard. The standard has been defined through an open process by a community of parallel computing vendors, computer scientists, and application developers.

The goal of the Message-Passing Interface :

1. The major goal is to develop a widely used standard for writing message-passing programs.
2. Design an application programming interface (not necessarily for compilers or a system implementation library).

RSCOE-IT

3. Allow efficient communication: Avoid memory-to-memory copying, allow over of computation and communication, and offered to communication co-processors, where available.
4. Allow for implementations that can be used in a heterogeneous environment.

An MPI program consists of autonomous processes, executing their own code, in an MIMD style. The codes executed by each process need not be identical. The processes communicate via calls to MPI communication primitives. Typically, each process executes in its own address space, although shared-memory implementations of MPI are possible. This document specifies the behaviour of a parallel program assuming that only MPI calls are used. The interaction of an MPI program with other possible means of communication, I/O, and process management is not specified. Unless otherwise stated in the specification of the standard, MPI places no requirements on the result of its interaction with external mechanisms that provide similar or equivalent functionality. This includes, but is not limited to, interactions with external mechanisms for process control, shared and remote memory access, _le system access and control, inter process communication, process signalling, and terminal I/O. High quality implementations should strive to make the results of such interactions intuitive to users, and attempt to document restrictions where deemed necessary. MPI provides the user with reliable message transmission. A message sent is always received correctly, and the user does not need to check for transmission errors, time-outs, or other error conditions. In other words, MPI does not provide mechanisms for dealing with failures in the communication system.

If the MPI implementation is built on an unreliable underlying mechanism, then it is the job of the implementor of the MPI subsystem to insulate the user from this unreliability, or to react unrecoverable errors as failures. Whenever possible, such failures will be reected as errors in the relevant communication call. Similarly, MPI itself provides no mechanisms for handling processor failures.

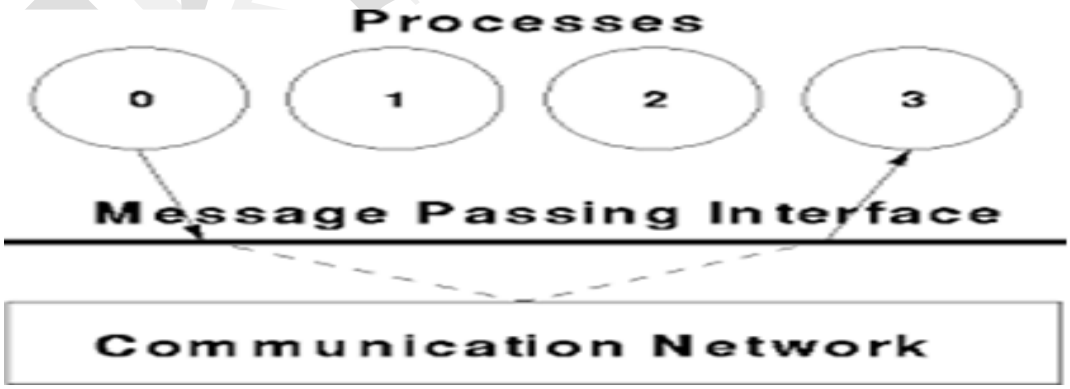


Figure: Message-Passing Interface Communication

- **MPI Data Type**

The Data Types available with MPI corresponding to c are listed in table 1.

Table 1: Predefined MPI data types corresponding to C data types

MPI Data Type	C / Fortran Data Type
MPI_CHAR	char
MPI_SHORT	signed short int
MPI_INT	signed int
MPI_LONG_LONG_INT	signed long long int
MPI_LONG	signed long int
MPI_LONG_LONG	signed long long int
MPI_UNSIGNED_CHAR	unsigned char
MPI_UNSIGNED_SHORT	unsigned short int
MPI_UNSIGNED_LONG	unsigned long int
MPI_UNSIGNED	unsigned int
MPI_FLOAT	float
MPI_DOUBLE	double

MPI Primitives:

Following are the some of primitives used in MPI for communication between the processes.

1. **MPI_Init :** Initialize or start MPI Communication
2. **MPI_Finalize:** Close or Exit MPI Communication
3. **MPI_Send() :** Mpi Blocking Send look like

send(dest, type, address, length)

Where:

- dest is an integer identi_er representing the process to receive the message.
- type is a nonnegative integer that the destination can use to selectively screen messages.
- Address describes a contiguous area in memory containing the message to be sent.

RSCOE-IT

4. **MPI_Send() :** Mpi Blocking Receive look like

MPI_RECV (buf, count, datatype, source, tag, comm, status)

Where:

- buf initial address of receive buffer .
- count number of elements in receive buffer.
- datatype is a datatype of each receive buffer element
- source rank of source or MPI_ANY_SOURCE
- tag message tag or MPI_ANY_TAG
- comm communicator
- status status object

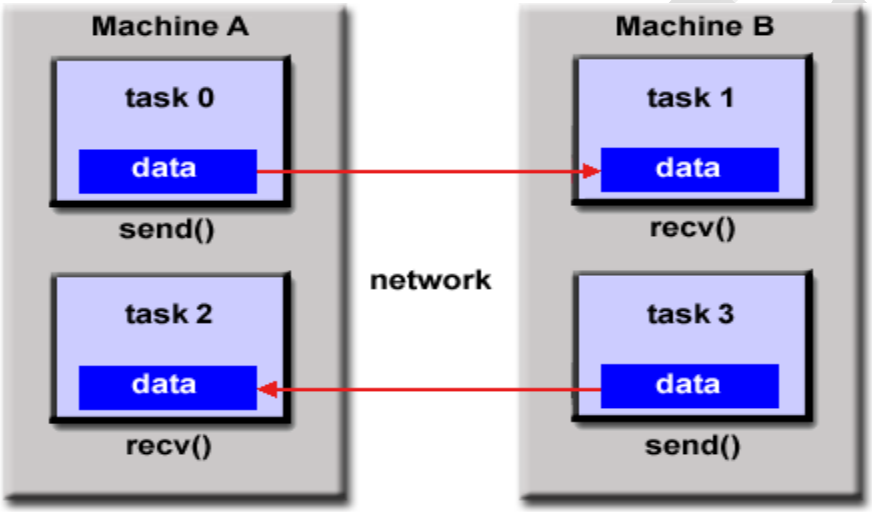


Figure: MPI Blocking Send and Receive Premitives.

Sending and receiving of messages by processes is the basic MPI communication mechanism. The basic point-to-point communication operations are send and receive. Their use is illustrated in the example below.

```
#include "mpi.h"
int main( int argc, char *argv[])
{
char message[20]; int myrank; MPI_Status status;
```

```
MPI_Init( &argc, &argv );
MPI_Comm_rank( MPI_COMM_WORLD, &myrank ); if (myrank == 0) /* code for process zero */
{
strcpy(message,"Hello, there");
MPI_Send(message, strlen(message)+1, MPI_CHAR, 1, 99, MPI_COMM_WORLD);
}
else if (myrank == 1) /* code for process one */
{
MPI_Recv(message, 20, MPI_CHAR, 0, 99, MPI_COMM_WORLD, &status); printf("received :%s:\n",
message);
}
MPI_Finalize(); return 0;
}
```

In this example, process zero (myrank = 0) sends a message to process one using the send operation MPI_SEND. The operation specifies a send buffer in the sender memory from which the message data is taken. In the example above, the send buffer consists of the storage containing the variable message in the memory of process zero. The location, size and type of the send buffer are specified
by the firrst three parameters of the send operation. The message sent will contain the 13 characters of this variable. In addition, the send operation associates an envelope with the message. This envelope specified the message destination and contains distinguishing information that can be used by the receive operation to select a particular message. The last three parameters of the send operation, along with the rank of the sender, specify the envelope for the message sent. Process one (myrank = 1) receives this message with the receive operation MPI_RECV. The message to be received is selected according to the value of its envelope, and the message data is stored into the receive buffer. In the example above, the receive buffer consists of the storage containing the string message in the memory of process one. The firrst three parameters of the receive operation specify the location, size and type of the receive buffer .The next three parameters are used for selecting the incoming message. The last parameter is used to return information on the message just received.

RSCOE-IT

CONCLUSION:

We have studied the all methods of communication between two or group of processes using MPI (Message Passing Interface). We implemented Message Passing Interface communication program which shows us the effective and successful communication between the group of process.

FAQs:

5. What is MPI?
6. What is are the Goals of MPI?
7. What are the Primitives of MPI?
8. What is Use of Blocking Send in MPI?
9. What is Use of Blocking Receive in MPI?
10. What are Data Types in MPI?
11. Write a Program in C for MPI Group communication?

RSCOE-IT

Experiment No. 4

Title: To Develop Distributed Applications Using Client-Server Communication through Java Sockets and RMI Techniques

1. Aim

To develop distributed applications using client-server communication by implementing communication programs based on Java Socket Programming and Remote Method Invocation (RMI) techniques.

2. Objectives

After completing this experiment, students will be able to:

1. Understand the concept of distributed applications.
2. Understand the client-server communication model.
3. Implement communication using TCP sockets in Java.
4. Develop a Java Socket Server and Socket Client.
5. Understand the architecture and working of Java RMI.
6. Develop and invoke remote methods using Java RMI.
7. Compare Socket-based communication with RMI-based communication.
8. Apply distributed communication concepts to real-world applications.

3. Prerequisite Knowledge

Students should have knowledge of:

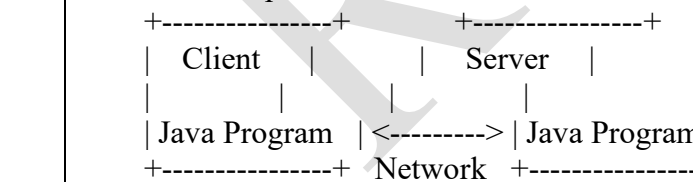
- Java programming
- Classes and objects
- Inheritance and interfaces
- Exception handling
- Input/Output streams
- Networking fundamentals
- IP address and port number
- Client-server architecture
- Multithreading basics

4. Theory

4.1 Distributed Application

A distributed application is an application whose components execute on multiple computers or processes and communicate with each other over a network.

For example:



The client sends a request to the server, and the server processes the request and sends a response.

---

5. Client-Server Model

The client-server model consists of two major components:

Client

The client:

- Initiates communication.

RSCOE-IT

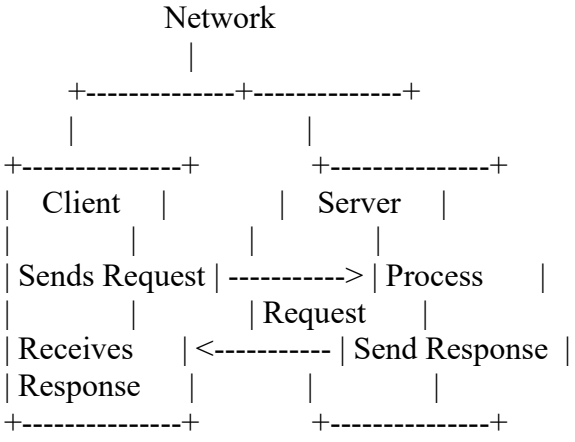
- Sends requests.
- Receives responses.

Server

The server:

- Waits for client requests.
- Accepts client connections.
- Processes requests.
- Sends responses.

Architecture



Part A – Java Socket Programming

6. Java Socket Programming

A socket is an endpoint for communication between two machines over a network.

Java provides the following important classes:

Class	Purpose
Socket	Establishes client-server connection
ServerSocket	Creates a server and listens for connections
InputStream	Reads data
OutputStream	Sends data
BufferedReader	Reads text efficiently
PrintWriter	Sends formatted text

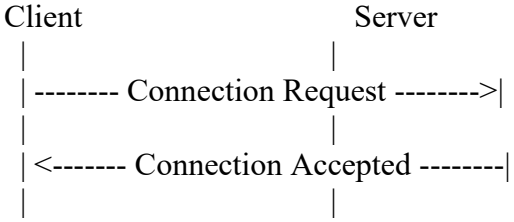
7. TCP Communication

For this experiment, we use TCP (Transmission Control Protocol).

TCP provides:

- Reliable communication
- Connection-oriented communication
- Ordered data delivery
- Error detection
- Retransmission of lost data

Basic TCP communication:



RSOE-IT

```
| ---------- Request ---------->|
|                               |
| <---------- Response ----------|
|                               |
```

8. Problem Statement – Socket

Develop a Java client-server application using TCP Socket Programming where:

1. The server listens on a specified port.
2. The client connects to the server.
3. The client sends a message.
4. The server receives the message.
5. The server processes the message.
6. The server sends a response.
7. The client displays the response.

9. Algorithm – Socket Server

Step 1 Start the server.
Step 2 Create a ServerSocket using a port number.
Step 3 Wait for a client connection using accept().
Step 4 Create input and output streams.
Step 5 Receive the client's message.
Step 6 Process the message.
Step 7 Send the response to the client.
Step 8 Close the connection.
Step 9 Stop the server.

10. Algorithm – Socket Client

Step 1 Start the client.
Step 2 Create a Socket using the server IP address and port number.
Step 3 Create input and output streams.
Step 4 Send a message to the server.
Step 5 Receive the server response.
Step 6 Display the response.
Step 7 Close the socket.

11. Java Socket Server Program

Save as:
SocketServer.java

```
import java.io.*;
import java.net.*;

public class SocketServer {

    public static void main(String[] args) {

        int port = 5000;

        try {
```

RSCOE-IT

```
        // Create server socket
        ServerSocket serverSocket =
                new ServerSocket(port);

        System.out.println(
                "Server started...");

        System.out.println(
                "Waiting for client connection...");

        // Accept client connection
        Socket socket =
                serverSocket.accept();

        System.out.println(
                "Client connected.");

        // Create input stream
        BufferedReader input =
                new BufferedReader(
                        new InputStreamReader(
                                socket.getInputStream()));

        // Create output stream
        PrintWriter output =
                new PrintWriter(
                        socket.getOutputStream(),
                        true);

        // Receive message
        String message =
                input.readLine();

        System.out.println(
                "Client says: " + message);

        // Send response
        output.println(
                "Message received successfully by Server.");

        // Close resources
        input.close();
        output.close();
        socket.close();
        serverSocket.close();

    } catch (IOException e) {

        System.out.println(
```

RSCOE-IT

```
                "Server Error: " + e);
        }
    }
}
```

12. Java Socket Client Program

Save as:

SocketClient.java

```
import java.io.*;
import java.net.*;

public class SocketClient {

    public static void main(String[] args) {

        String serverAddress = "localhost";
        int port = 5000;

        try {

            // Connect to server
            Socket socket =
                new Socket(serverAddress, port);

            System.out.println(
                "Connected to server.");

            // Create output stream
            PrintWriter output =
                new PrintWriter(
                    socket.getOutputStream(),
                    true);

            // Create input stream
            BufferedReader input =
                new BufferedReader(
                    new InputStreamReader(
                        socket.getInputStream()));

            // Send message
            output.println(
                "Hello Server, this is the Client.");

            // Receive response
            String response =
                input.readLine();

            System.out.println(
                "Server Response: " + response);
```

RSCOE-IT

```
            // Close resources
            output.close();
            input.close();
            socket.close();

        } catch (IOException e) {

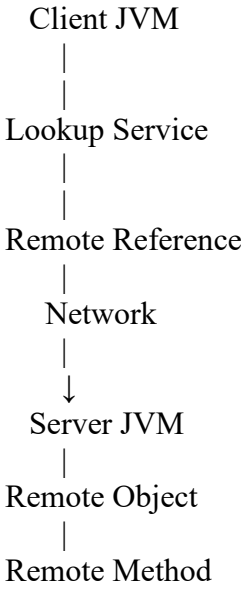
            System.out.println(
                  "Client Error: " + e);
        }
    }
}
```

13. Socket Execution
Terminal 1 – Server
Compile:
javac SocketServer.java
Run:
java SocketServer
Output:
Server started...
Waiting for client connection...
Terminal 2 – Client
Compile:
javac SocketClient.java
Run:
java SocketClient
Client output:
Connected to server.
Server Response: Message received successfully by Server.
Server output:
Server started...
Waiting for client connection...
Client connected.
Client says: Hello Server, this is the Client.

Part B – Java RMI
14. Java RMI
RMI stands for Remote Method Invocation.
Java RMI allows an object running in one JVM to invoke a method on an object running in another JVM.
In simple words:
RMI allows a Java program to call a method of an object located on another machine as if it were a local method.

RSCOE-IT

15. RMI Architecture



16. Components of RMI

The major components are:

1. Remote Interface
2. Remote Object Implementation
3. RMI Registry
4. RMI Server
5. RMI Client

17. Problem Statement – RMI

Develop a Java RMI application where:

- The server provides a remote method.
- The client looks up the remote object.
- The client invokes the remote method.
- The server executes the method.
- The result is returned to the client.

We will implement a simple Remote String Analyzer.
The client sends a string to the server, and the server returns the number of characters in the string.

18. Step 1 – Remote Interface

Save as:
StringAnalyzer.java

```
import java.rmi.*;

public interface StringAnalyzer extends Remote {

   int countCharacters(String str)
         throws RemoteException;
```

RSCOE-IT

}
Explanation
extends Remote
indicates that this interface contains methods that can be invoked remotely.

int countCharacters(String str)
This is the remote method.

throws RemoteException
Remote communication can fail because of network or server problems, so RMI methods must handle RemoteException.

19. Step 2 – Remote Object Implementation
Save as:
StringAnalyzerImpl.java

```
import java.rmi.*;
import java.rmi.server.*;

public class StringAnalyzerImpl
      extends UnicastRemoteObject
      implements StringAnalyzer {

   public StringAnalyzerImpl()
         throws RemoteException {
      super();
   }

   @Override
   public int countCharacters(String str)
         throws RemoteException {

      return str.length();
   }
}
```

20. Explanation
UnicastRemoteObject
extends UnicastRemoteObject
makes the object available for remote invocation.

Implement Interface
implements StringAnalyzer
means that the class provides the implementation of the remote method.

Remote Method
public int countCharacters(String str)
receives a string and returns its length.
For example:
Input: Distributed

RSCOE-IT

Output: 11

23. Step 3 – RMI Server
Save as:
RMIServer.java

```
import java.rmi.registry.*;

public class RMIServer {

   public static void main(String[] args) {

      try {

         // Create remote object
         StringAnalyzerImpl obj =
               new StringAnalyzerImpl();

         // Create RMI Registry
         Registry registry =
               LocateRegistry.createRegistry(1099);

         // Bind remote object
         registry.rebind(
               "StringAnalyzer",
               obj);

         System.out.println(
               "RMI Server is ready...");

      } catch (Exception e) {

         System.out.println(
               "Server Error: " + e);
      }
   }
}
```

21. Important RMI Server Concepts
Create Remote Object

```
StringAnalyzerImpl obj =
      new StringAnalyzerImpl();
```

Creates the remote object.

Create Registry

```
LocateRegistry.createRegistry(1099);
```

Creates the RMI registry on port 1099.
The registry stores references to remote objects.

Bind Object

RSCOE-IT

```
registry.rebind(
        "StringAnalyzer",
        obj);
```

Registers the remote object with the name:
StringAnalyzer
The client uses this name to locate the remote object.

22. Step 4 – RMI Client
Save as:
RMIClient.java

```
import java.rmi.registry.*;

public class RMIClient {

    public static void main(String[] args) {

        try {

            // Connect to RMI Registry
            Registry registry =
                    LocateRegistry.getRegistry(
                            "localhost",
                            1099);

            // Lookup remote object
            StringAnalyzer analyzer =
                    (StringAnalyzer)
                    registry.lookup(
                            "StringAnalyzer");

            // Invoke remote method
            String text =
                    "Parallel Computing";

            int result =
                    analyzer.countCharacters(text);

            System.out.println(
                    "String: " + text);

            System.out.println(
                    "Number of Characters: "
                    + result);

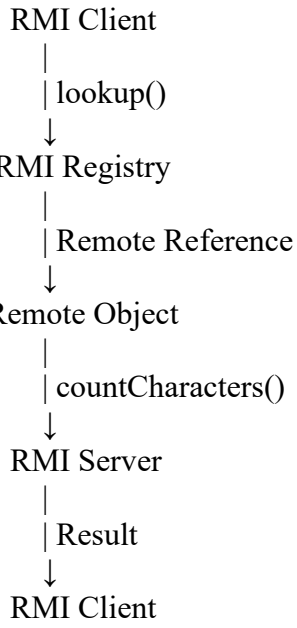
        } catch (Exception e) {

            System.out.println(
                    "Client Error: " + e);
        }
```

RSCOE-IT

```
    PDC Lab
    }
}
```

23. RMI Execution Flow



24. Compilation and Execution
Compile all files:
javac *.java
Start the RMI server:
java RMIServer
Expected output:
RMI Server is ready...
Open another terminal and run:
java RMIClient
Expected output:
String: Parallel Computing
Number of Characters: 19
The exact count includes the space between the words.

25. Comparison: Socket vs RMI

Feature	Java Socket	Java RMI
Communication	Network-based	Network-based
Programming level	Low-level	Higher-level
Communication	Stream/message based	Method invocation
Main classes	Socket, ServerSocket	Remote, UnicastRemoteObject
Data handling	Programmer manages streams	RMI handles remote invocation
Object-oriented	Less abstract	Highly object-oriented
Registry	Not required	RMI Registry required

RSCOE-IT

Ease of implementation	Moderate	Easier for Java-to-Java applications
Protocol details	Developer handles more details	RMI abstracts many details
Best suited for	General network communication	Distributed Java applications

26. Socket vs RMI Architecture

Socket

Client
|
| Socket
↓
Server
|
| Process data
↓
Response

RMI

Client
|
| Remote Method Call
↓
RMI Registry
|
↓
Remote Object
|
| Execute Method
↓
Return Result

27. Applications

Socket Programming

- Chat applications
- File transfer
- Multiplayer games
- Client-server applications
- Network monitoring
- Messaging applications

RMI

- Distributed Java applications
- Remote object management
- Distributed computing
- Enterprise applications
- Remote service invocation

Conclusion

Thus, client-server communication programs were successfully developed using Java Socket Programming and Java RMI, demonstrating network communication and remote method invocation in a distributed computing environment.

RSCOE-IT

Experiment No. 5

Title: To Develop a Distributed Application Using Messaging System in Publish-Subscribe Paradigm

Aim

To develop a messaging-based application using the Publish-Subscribe (Pub-Sub) paradigm and demonstrate concurrent communication between a Publisher and multiple Subscribers using OpenMP.

Objectives

After completing this experiment, students will be able to:

1. Understand the Publish-Subscribe messaging paradigm.
2. Understand the role of a Publisher, Topic, and Subscriber.
3. Implement parallel execution using OpenMP.
4. Use OpenMP parallel sections for concurrent execution.
5. Use critical sections to protect shared resources.
6. Demonstrate communication between multiple subscribers through a shared topic.
7. Understand synchronization in parallel applications.

Prerequisite Knowledge

Students should have basic knowledge of:

- C programming
- Functions and arrays
- Strings in C
- Process and thread concepts
- Parallel computing
- OpenMP
- Shared-memory programming
- Critical sections and synchronization

Theory

Messaging System

A messaging system allows different components of an application to communicate by exchanging messages.

Instead of one component directly calling another component, a message is sent through a messaging mechanism.

For example:

Application A

|

| Message

↓

Messaging System

|

↓

Application B

Messaging systems are commonly used in distributed applications because they provide a mechanism for exchanging information between independent components.

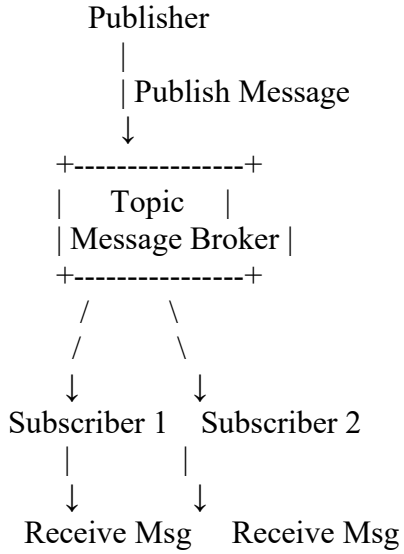
Publish-Subscribe Paradigm

The Publish-Subscribe (Pub-Sub) model is a messaging communication pattern in which:

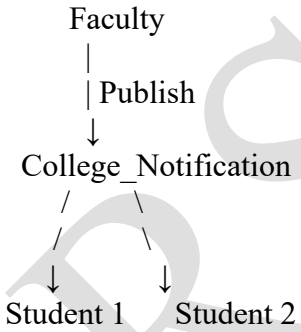
RSCOE-IT

- A Publisher generates messages.
- A Topic acts as a communication channel.
- A Subscriber receives messages from the topic.

The publisher does not need to know who the subscribers are.
Similarly, subscribers do not need to know who generated the message.
Basic Architecture



Real-World Example
Consider a college notification system.
A faculty member publishes:
"PDC Practical Exam will be conducted on Friday."
The message is published to a topic:
College_Notification
Multiple students subscribed to this topic receive the message.



The publisher does not need to send the message separately to each student.

Publish-Subscribe vs Point-to-Point

Point-to-Point	Publish-Subscribe
One sender communicates with one receiver	One publisher can communicate with many subscribers
Message generally goes to one consumer	Message can be delivered to multiple subscribers
Direct communication	Topic-based communication
Suitable for one-to-one communication	Suitable for one-to-many communication
Consumer receives a particular message	All interested subscribers receive the message

8. OpenMP

RSCOE-IT

OpenMP (Open Multi-Processing) is an API used for parallel programming in shared-memory systems.
OpenMP allows a program to create multiple threads that can execute tasks concurrently.
Important OpenMP directives used in this experiment are:
#pragma omp parallel sections
and
#pragma omp critical
OpenMP parallel sections
The directive:
#pragma omp parallel sections
allows different sections of a program to execute concurrently.
Example:

```
#pragma omp parallel sections
{
   #pragma omp section
   {
      // Publisher
   }

   #pragma omp section
   {
      // Subscriber 1
   }

   #pragma omp section
   {
      // Subscriber 2
   }
}
```

Different sections can be executed by different threads.

OpenMP Critical Section
The directive:
#pragma omp critical
ensures that only one thread at a time executes a particular block of code.
Example:

```
#pragma omp critical
{
   messageCount++;
}
```

This is important when multiple threads access or modify shared data.

Problem Statement
Develop a C program using OpenMP to implement a simple Publish-Subscribe Messaging System.
The application should:

1. Create a publisher.
2. Create a shared topic.

RSCOE-IT

3. Publish multiple messages.
4. Create at least two subscribers.
5. Allow subscribers to receive the published messages.
6. Use OpenMP for concurrent execution.
7. Use synchronization to protect shared resources.

Algorithm
Step 1
Start the program.
Step 2
Create a shared topic to store messages.
Step 3
Create a publisher.
Step 4
Create multiple messages.
Step 5
Publish messages into the shared topic.
Step 6
Create Subscriber 1 and Subscriber 2.
Step 7
Allow subscribers to access the topic concurrently.
Step 8
Display the messages received by each subscriber.
Step 9
Use critical to protect shared data.
Step 10
Terminate the program.

Program

```
#include <stdio.h>
#include <omp.h>
#include <string.h>

#define MAX_MESSAGES 5

// Shared topic
char topic[MAX_MESSAGES][100];

// Number of messages published
int messageCount = 0;

int main()
{
   printf("==============================================\n");
   printf(" Publish-Subscribe Messaging System\n");
   printf(" Using OpenMP\n");
   printf("==============================================\n");

   /*
```

RSCOE-IT

```
 * Publisher
 */
#pragma omp parallel sections
{
   #pragma omp section
   {
      char *messages[] =
      {
         "PDC Lecture at 9:00 AM",
         "Submit PDC Journal",
         "PDC Assignment Due Today",
         "Practical Exam on Friday",
         "Attend PDC Lecture Regularly"
      };

      for (int i = 0; i < MAX_MESSAGES; i++)
      {
         #pragma omp critical
         {
            strcpy(topic[messageCount],
                  messages[i]);

            messageCount++;

            printf("\nPublisher: %s",
                  messages[i]);
         }
      }
   }

   /*
    * Subscriber 1
    */
   #pragma omp section
   {
      printf("\nSubscriber 1 is connected to the topic.\n");
   }

   /*
    * Subscriber 2
    */
   #pragma omp section
   {
      printf("Subscriber 2 is connected to the topic.\n");
   }
}

/*
 * Subscribers receive messages
```

RSCOE-IT

```
PDC Lab
 */
#pragma omp parallel sections
{
   #pragma omp section
   {
      printf("\n\n========== Subscriber 1 ==========\n");

      for (int i = 0; i < messageCount; i++)
      {
         printf("Subscriber 1 received: %s\n",
               topic[i]);
      }
   }

   #pragma omp section
   {
      printf("\n========== Subscriber 2 ==========\n");

      for (int i = 0; i < messageCount; i++)
      {
         printf("Subscriber 2 received: %s\n",
               topic[i]);
      }
   }
}

printf("\nAll messages delivered successfully.\n");

return 0;
}
```

Sample Output
The exact ordering of thread-related output may vary.

```
==========================================
 Publish-Subscribe Messaging System
 Using OpenMP
==========================================

Subscriber 1 is connected to the topic.
Subscriber 2 is connected to the topic.

Publisher: PDC Lecture at 9:00 AM
Publisher: Submit PDC Journal
Publisher: PDC Assignment Due Today
Publisher: Practical Exam on Friday
Publisher: Attend PDC Lecture Regularly

========== Subscriber 1 ==========
```

RSCOE-IT

Subscriber 1 received: PDC Lecture at 9:00 AM
Subscriber 1 received: Submit PDC Journal
Subscriber 1 received: PDC Assignment Due Today
Subscriber 1 received: Practical Exam on Friday
Subscriber 1 received: Attend PDC Lecture Regularly

========== Subscriber 2 ==========
Subscriber 2 received: PDC Lecture at 9:00 AM
Subscriber 2 received: Submit PDC Journal
Subscriber 2 received: PDC Assignment Due Today
Subscriber 2 received: Practical Exam on Friday
Subscriber 2 received: Attend PDC Lecture Regularly

All messages delivered successfully.

Compilation and Execution
Linux / Ubuntu
Save the program as:
pubsub.c
Compile:
gcc -fopenmp pubsub.c -o pubsub
Run:
./pubsub
Windows using GCC/MinGW
gcc -fopenmp pubsub.c -o pubsub.exe
Run:
pubsub.exe
Advantages of Publish-Subscribe

1. Loose coupling between publisher and subscriber.
2. One message can be delivered to multiple subscribers.
3. Publishers do not need to know subscriber details.
4. Easy to add new subscribers.
5. Suitable for event-driven applications.
6. Supports asynchronous communication.

Limitations of This OpenMP Demonstration
Students should understand an important distinction:
OpenMP is primarily a shared-memory parallel programming API, whereas a real distributed Publish-Subscribe system normally uses a message broker and network communication.
This lab program uses OpenMP to demonstrate the Pub-Sub concept and concurrent publisher/subscriber processing, rather than implementing a production distributed messaging broker.
For a true distributed implementation, technologies such as MQTT, Apache Kafka, RabbitMQ, or Java Message Service (JMS) can be used.

Conclusion
Thus, a messaging-based application using the Publish-Subscribe paradigm was developed successfully using OpenMP, demonstrating concurrent message publishing and reception by multiple subscribers with appropriate synchronization of shared resources.

Experiment No. 6

RSCOE-IT

Aim

To create a simple web service using Java and develop a distributed client application that consumes the web service over a network.

Objectives

After completing this experiment, students will be able to:

- Understand the concept of Web Services.
- Understand the basic architecture of distributed applications.
- Create a simple SOAP-based web service.
- Publish a web service.
- Generate and understand a WSDL description.
- Develop a client application to consume the web service.
- Understand request-response communication between distributed applications.
- Analyze the role of networking in distributed computing.

Theory

Introduction to Web Services

A Web Service is a software component that provides functionality over a network and allows other applications to communicate with it using standard protocols.

A web service enables applications developed using different programming languages or running on different platforms to communicate with each other.

For example:

Client Application

|

| Request

↓

Network

|

↓

Web Service

|

| Response

↓

Client Application

The client does not need to know how the service is internally implemented. It only needs to know:

- Service location
- Operations provided
- Input parameters
- Output format
- Communication protocol

Distributed Application

A distributed application is an application in which components execute on different computers or processes and communicate through a network.

In this experiment:

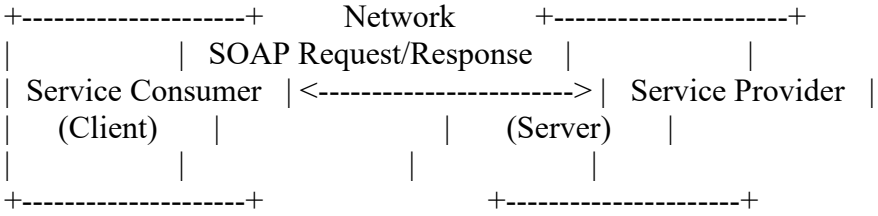
Service Provider

Provides the web service.

Service Consumer

Consumes the service.

RSCOE-IT



SOAP Web Service

SOAP (Simple Object Access Protocol) is a protocol used for exchanging structured information between applications over a network.

SOAP messages are generally represented using XML.

A SOAP communication consists of:

1. SOAP Request
2. Service Processing
3. SOAP Response

Example:

Client
|
| SOAP Request
| "add(10,20)"
↓
Web Service
|
| Processing
↓
SOAP Response
|
| "30"
↓
Client

WSDL

WSDL (Web Services Description Language) describes a web service and specifies how a client can communicate with it.

WSDL generally describes:

- Service operations
- Input parameters
- Output parameters
- Data types
- Service endpoint
- Communication binding

Conceptually:

Client
|
| Reads WSDL
↓
Understands available operations

RSCOE-IT

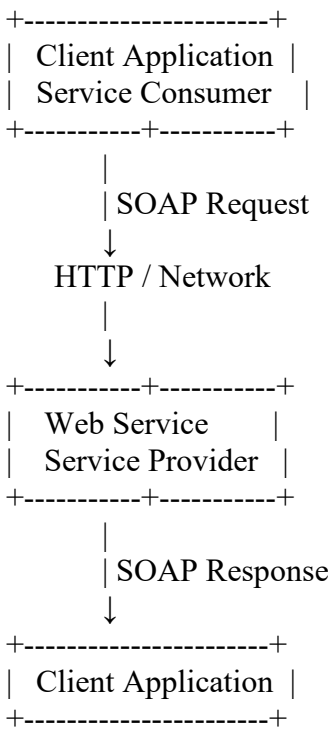
| Sends Request

↓

Web Service

Architecture of the Experiment

Distributed Environment



Software Requirements

Hardware

- Computer/Laptop
- Minimum 4 GB RAM
- Network connection

Software

- Ubuntu/Linux or Windows
- JDK
- Java compiler
- Text Editor / VS Code / IntelliJ IDEA / Eclipse
- Terminal

Important Note Regarding Java Versions

The following experiment uses the JAX-WS SOAP approach.

JAX-WS was removed from the standard JDK beginning with JDK 11, so students using modern JDK

RSCOE-IT

versions should use a JAX-WS implementation such as Metro/Jakarta XML Web Services or perform the experiment using a lab-provided JDK/project configuration.
For a teaching laboratory, the simplest approach is to use the JDK/project configuration supplied by the faculty.

Algorithm
Part A: Web Service Provider
1. Start.
2. Create a Java interface for the web service.
3. Define the service operation.
4. Create a class implementing the service.
5. Annotate the interface/class as a web service.
6. Publish the service at an endpoint.
7. Start the service.
8. Make the service available to clients.
9. Stop.

Part B: Web Service Consumer
1. Start.
2. Identify the service endpoint.
3. Obtain the service description/WSDL.
4. Create a client application.
5. Create a service proxy/stub.
6. Invoke the remote operation.
7. Receive the response.
8. Display the result.
9. Stop.

Web Service to be Developed
In this experiment, we develop a simple Calculator Web Service.
The service provides the following operations:
add(a,b)
subtract(a,b)
multiply(a,b)
For example:
add(20,10)
returns:
30

Program 1 – Web Service Interface
Create a file:
CalculatorService.java

```
import javax.jws.WebMethod;
import javax.jws.WebService;

@WebService
public interface CalculatorService {

   @WebMethod
```

RSCOE-IT

```
    int add(int a, int b);

    @WebMethod
    int subtract(int a, int b);

    @WebMethod
    int multiply(int a, int b);
}
```

Explanation
@WebService
@WebService
Marks the interface as a web service.
@WebMethod
@WebMethod
Indicates that a method should be exposed as a web-service operation.

Program 2 – Web Service Implementation
Create:
CalculatorServiceImpl.java

```
import javax.jws.WebService;

@WebService(endpointInterface = "CalculatorService")
public class CalculatorServiceImpl
        implements CalculatorService {

    @Override
    public int add(int a, int b) {
        return a + b;
    }

    @Override
    public int subtract(int a, int b) {
        return a - b;
    }

    @Override
    public int multiply(int a, int b) {
        return a * b;
    }
}
```

Explanation
The implementation class contains the actual business logic.
For example:

```
public int add(int a, int b) {
    return a + b;
}
```

When the client requests:

RSCOE-IT

add(20,10)
the server calculates:
20 + 10 = 30
and returns 30 to the client.

Program 3 – Publish the Web Service
Create:
CalculatorPublisher.java

```
import javax.xml.ws.Endpoint;

public class CalculatorPublisher {

   public static void main(String[] args) {

      String url =
         "http://localhost:8080/calculator";

      Endpoint.publish(
         url,
         new CalculatorServiceImpl()
      );

      System.out.println(
         "Calculator Web Service Started..."
      );

      System.out.println(
         "Service URL: " + url
      );

      System.out.println(
         "WSDL URL: " + url + "?wsdl"
      );
   }
}
```

Explanation of Publisher Program
The statement:

```
Endpoint.publish(
   url,
   new CalculatorServiceImpl()
);
```

publishes the web service at:
http://localhost:8080/calculator
The WSDL can be accessed conceptually at:
http://localhost:8080/calculator?wsdl
The WSDL provides the description required by clients to understand the service.

RSCOE-IT

Expected Server Output
After running the publisher:
Calculator Web Service Started...

Service URL:
http://localhost:8080/calculator

WSDL URL:
http://localhost:8080/calculator?wsdl
The server must remain running while the client application is being executed.

Service Consumer
A client application can consume the service using a generated service proxy based on the WSDL.
Conceptually:
WSDL
|
↓
Generate Client Proxy
|
↓
Client Application
|
↓
Invoke add()
|
↓
SOAP Request
|
↓
Web Service
|
↓
SOAP Response
|
↓
Client

Client Program
The exact generated proxy class names depend on the JAX-WS tooling/version being used.
A typical client structure is:

```
public class CalculatorClient {

   public static void main(String[] args) {

      CalculatorService service =
         new CalculatorService();

      CalculatorServicePortType port =
         service.getCalculatorServicePort();
```

RSCOE-IT

```
        int result = port.add(20, 10);

        System.out.println(
            "Addition = " + result
        );
    }
}
```

Important: The names CalculatorService and CalculatorServicePortType in the client are generated from the WSDL and may differ depending on the generated client artifacts. Students should use the actual classes generated by their JAX-WS tool.

Client Execution
Suppose the client sends:
a = 20
b = 10
The request is sent to the remote service.
The service performs:
20 + 10
The result is:
30

Expected Client Output
Connected to Calculator Web Service

20 + 10 = 30

20 - 10 = 10

20 * 10 = 200

Request-Response Communication
The communication can be represented as:
Client
 |
 | Request:
 | add(20,10)
 ↓
Network
 |
 ↓
Web Service

RSCOE-IT

```
  |
  | Calculate
  | 20 + 10
  ↓
Network
  |
  | Response:
  | 30
  ↓
Client
```

Distributed Nature of the Application
The client and server can run on different computers.
Computer 1
Client Application
Computer 2
Web Service
Communication occurs through the network:

```
+----------------+           +----------------+
| Client Machine |           | Server Machine |
|                |           |                |
| Java Client    | ----------> | Web Service    |
|                |  Network  |                |
+----------------+           +----------------+
```

If the service is running on another computer, the endpoint can use the server's IP address instead of localhost.
For example:
http://192.168.x.x:8080/calculator
The exact IP address depends on the laboratory network.

Execution Steps in Ubuntu Linux
Step 1: Check Java
Open Terminal:
Ctrl + Alt + T
Check Java:
java -version
Check compiler:
javac -version

Step 2: Create Project Directory
mkdir WebServiceLab
cd WebServiceLab

Step 3: Create Java Files
Create:
nano CalculatorService.java

RSCOE-IT

Create:
nano CalculatorServiceImpl.java
Create:
nano CalculatorPublisher.java
Enter the respective programs.

Compile
For a compatible JAX-WS/JDK lab configuration, compile the source files according to the libraries configured for the experiment.
For a legacy JDK configuration where JAX-WS is included:
javac *.java
For a modern JDK, add the JAX-WS/Metro dependencies configured by your lab environment.

Run the Service
java CalculatorPublisher
Expected output:
Calculator Web Service Started...

Service URL:
http://localhost:8080/calculator

WSDL URL:
http://localhost:8080/calculator?wsdl
Keep this terminal running.

Verify the Service
Open a web browser and enter:
http://localhost:8080/calculator?wsdl
If the service is running correctly, the browser should display the XML-based WSDL document.

Important Concepts Demonstrated
This experiment demonstrates the following distributed-computing concepts:

Concept	Demonstration
Service Provider	Calculator Web Service
Service Consumer	Java Client
Remote Communication	SOAP over HTTP
Service Description	WSDL
Request/Response	Calculator operation
Distributed Application	Client and server
Network Communication	HTTP
Loose Coupling	Client uses service interface
Platform Independence	Standard Web Service protocols

Result
The Calculator Web Service was created successfully and a distributed client application was developed to consume the service. The experiment demonstrated service-oriented communication, WSDL-based service description, SOAP request-response communication, and distributed application development.

NCOE-IT

RSCOE-IT

Experiment No. 7

Title

To Develop a Distributed Application with CORBA Program Using Java IDL

1. Aim

To study the Common Object Request Broker Architecture (CORBA) and develop a simple distributed application using Java IDL, demonstrating communication between a client and a remote server object.

Learning Objectives

After completing this experiment, students will be able to:

1. Explain the concept of CORBA.
2. Understand distributed object communication.
3. Explain the role of an Object Request Broker.
4. Understand CORBA Interface Definition Language (IDL).
5. Define a remote interface using IDL.
6. Generate Java bindings from an IDL file.
7. Develop a CORBA server application.
8. Develop a CORBA client application.
9. Register and locate a remote CORBA object.
10. Invoke methods on a remote object from a client application.

Prerequisites

Students should have basic knowledge of:

- Java programming
- Object-oriented programming
- Client-server architecture
- Distributed systems
- Remote procedure call concepts
- Exception handling in Java
- Compilation and execution of Java programs
- Basic networking concepts

Software Requirements

Software

- Java Development Kit (JDK)
- Java IDL/CORBA tools where supported
- Linux/Ubuntu or Windows
- Terminal/Command Prompt
- Text editor or IDE

Hardware

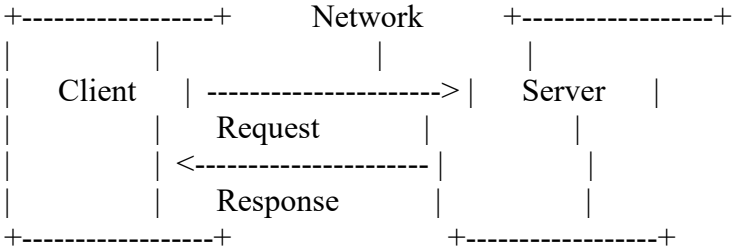
- Computer/Laptop
- Minimum 4 GB RAM recommended
- Network connectivity for multi-machine execution

RSCOE-IT

Introduction to Distributed Applications

A distributed application is an application in which different components execute on different computers or processes and communicate through a network.

For example:



The client requests a service, and the server provides the requested service.

Distributed applications provide:

- Resource sharing
- Remote service access
- Scalability
- Location transparency
- Improved resource utilization
- Interoperability

What is CORBA?

CORBA stands for:

Common Object Request Broker Architecture

CORBA is a standard architecture developed by the Object Management Group (OMG) for enabling objects written in different programming languages and running on different machines to communicate with each other.

CORBA follows the concept of distributed objects.

A client can invoke a method on a remote object almost as if the object were located locally.

Need for CORBA

In a distributed environment:

- Client and server may run on different machines.
- They may use different programming languages.
- Objects may be located at different network locations.
- Communication details should be hidden from the application programmer.

CORBA provides an abstraction layer between the client and server.

Without CORBA

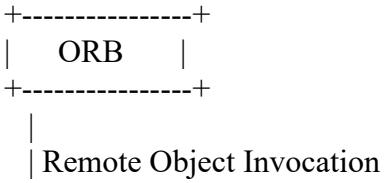
Client
|
| Network Programming
|
Server

The programmer must handle many communication details.

With CORBA

Client
|
| Object Request
v

RSCOE-IT

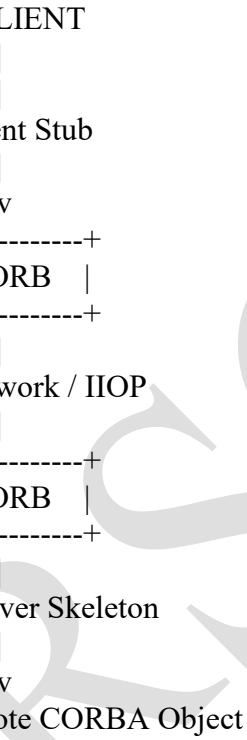


The ORB handles much of the communication infrastructure.

Object Request Broker (ORB)
ORB stands for Object Request Broker.
The ORB is the core component of CORBA.
It enables a client to communicate with a remote object without knowing the details of:

- Object location
- Communication protocol
- Server implementation
- Network communication
- Object activation

Basic ORB Architecture



Important CORBA Components
Client
The client requests services from a remote CORBA object.

Server
The server creates and provides the implementation of the CORBA object.

ORB
The Object Request Broker manages communication between the client and remote object.

RSCOE-IT

IDL
Interface Definition Language (IDL) is used to define the interface of a distributed object independently of the programming language.
Example:

```
interface Calculator
{
   long add(in long a, in long b);
};
```

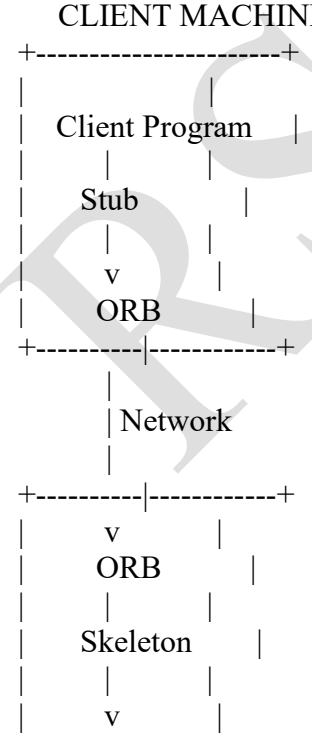
The IDL defines what operations are available but does not define their implementation.

Stub
The client-side stub provides a local representation of the remote object.
The client invokes methods through the stub.

Skeleton
The server-side skeleton receives requests from the ORB and forwards them to the actual implementation object.

Naming Service
The CORBA Naming Service allows clients to locate remote objects by name.
For example:
Calculator
The client can request the object named Calculator from the Naming Service.

CORBA Communication Model
The overall communication can be represented as:



RSCOE-IT

```
| Remote Object       |
|                     |
|   SERVER MACHINE    |
+---------------------+
```

CORBA IDL
IDL means Interface Definition Language.
IDL provides a language-independent description of an object's interface.
For example:

```
module CalculatorApp
{
   interface Calculator
   {
      long add(in long a, in long b);
      long subtract(in long a, in long b);
   };
};
```

Here:

- module groups related interfaces.
- interface defines a remote object.
- long represents an integer type.
- in indicates an input parameter.
- add() and subtract() are remote operations.

Experiment Problem Statement
Develop a distributed Calculator Application using CORBA and Java IDL.
The application should:

1. Define a Calculator interface using IDL.
2. Generate Java classes from the IDL.
3. Implement the Calculator interface on the server.
4. Register the object with the CORBA Naming Service.
5. Develop a client program.
6. Locate the remote Calculator object.
7. Invoke remote methods.
8. Display the result on the client.

The calculator will perform:

- Addition
- Subtraction
- Multiplication
- Division

Step 1 – Create the IDL File
Create a file named:
Calculator.idl
IDL Program

```
module CalculatorApp
{
   interface Calculator
   {
```

RSCOE-IT

```
    long add(in long a, in long b);
    long subtract(in long a, in long b);
    long multiply(in long a, in long b);
    double divide(in long a, in long b);
  };
};
```

Explanation of IDL Program
Module
module CalculatorApp
The module provides a namespace for the interface.
Interface
interface Calculator
Defines the remote object interface.
Addition
long add(in long a, in long b);
Accepts two integers and returns their sum.
Subtraction
long subtract(in long a, in long b);
Returns the difference.
Multiplication
long multiply(in long a, in long b);
Returns the product.
Division
double divide(in long a, in long b);
Returns the division result as a floating-point value.

Step 2 – Generate Java Classes
Historically, Java IDL/CORBA tools included the idlj compiler.
For JDK versions that provide the legacy Java IDL tool, the command is:
idlj -fall Calculator.idl
This generates Java classes and interfaces required for CORBA communication.
Typical generated components include classes/interfaces corresponding to:
Calculator.java
CalculatorHelper.java
CalculatorHolder.java
CalculatorOperations.java
CalculatorPOA.java
The exact generated files can vary according to the JDK/tool version.

Important JDK Compatibility Note
Students must verify the Java version before executing this experiment.
The traditional Java CORBA implementation and tools such as idlj were included in older JDK releases.
CORBA modules and Java IDL tools were removed from the standard JDK beginning with JDK 11.
Therefore, if the following command is unavailable:
idlj
the experiment should be executed using:

- the JDK version supplied by the laboratory, or

RSCOE-IT

- a compatible CORBA implementation/project configuration such as an appropriate legacy CORBA/JDK setup.

Do not assume that idlj will be available in a modern JDK installation.

Step 3 – Implement the Server Object
Create:
CalculatorImpl.java
Program

```
import CalculatorApp.*;

public class CalculatorImpl extends CalculatorPOA
{
   public int add(int a, int b)
   {
      return a + b;
   }

   public int subtract(int a, int b)
   {
      return a - b;
   }

   public int multiply(int a, int b)
   {
      return a * b;
   }

   public double divide(int a, int b)
   {
      if (b == 0)
         return 0;

      return (double)a / b;
   }
}
```

Explanation of Server Implementation
The class:
public class CalculatorImpl extends CalculatorPOA
implements the CORBA interface generated from IDL.
The methods:
add()
subtract()
multiply()
divide()
provide the actual business logic.
The client does not directly execute these methods. Instead, the request travels through the CORBA

RSCOE-IT

infrastructure.

Step 4 – Create CORBA Server
Create:
CalculatorServer.java
Program

```
import CalculatorApp.*;
import org.omg.CosNaming.*;
import org.omg.CosNaming.NamingContextPackage.*;
import org.omg.CORBA.*;

public class CalculatorServer
{
   public static void main(String args[])
   {
      try
      {
         ORB orb = ORB.init(args, null);

         CalculatorImpl calculatorImpl =
               new CalculatorImpl();

         orb.connect(calculatorImpl);

         org.omg.CORBA.Object objRef =
               orb.resolve_initial_references("NameService");

         NamingContextExt ncRef =
               NamingContextExtHelper.narrow(objRef);

         String name = "Calculator";

         org.omg.CORBA.Object ref =
               ncRef.resolve_str(name);

         if (ref == null)
         {
            ncRef.rebind(
               ncRef.to_name(name),
               calculatorImpl._this(orb)
            );
         }

         else
         {
            ncRef.rebind(
               ncRef.to_name(name),
               calculatorImpl._this(orb)
            );
```

RSCOE-IT

```
        }

        System.out.println(
                "Calculator Server Ready..."
        );

        orb.run();
      }
      catch (Exception e)
      {
        System.out.println(
                "Server Error: " + e
        );

        e.printStackTrace();
      }
    }
}
```

Explanation of Server Program
Initialize ORB
ORB orb = ORB.init(args, null);
Initializes the CORBA Object Request Broker.
Create Server Object
CalculatorImpl calculatorImpl =
new CalculatorImpl();
Creates the actual implementation object.
Obtain Naming Service
orb.resolve_initial_references("NameService");
Obtains a reference to the CORBA Naming Service.
Narrow the Reference
NamingContextExt ncRef =
NamingContextExtHelper.narrow(objRef);
Converts the generic object reference into a Naming Context reference.
Register Object

```
ncRef.rebind(
  ncRef.to_name("Calculator"),
  calculatorImpl._this(orb)
);
```

Registers the Calculator object with the name:
Calculator
Start ORB
orb.run();
Keeps the server running and ready to receive client requests.

Step 5 – Create Client Program
Create:
CalculatorClient.java
Program

RSCOE-IT

```
import CalculatorApp.*;
import org.omg.CosNaming.*;
import org.omg.CORBA.*;

public class CalculatorClient
{
   public static void main(String args[])
   {
      try
      {
         ORB orb = ORB.init(args, null);

         org.omg.CORBA.Object objRef =
               orb.resolve_initial_references(
                     "NameService");

         NamingContextExt ncRef =
               NamingContextExtHelper.narrow(objRef);

         Calculator calculator =
               CalculatorHelper.narrow(
                  ncRef.resolve_str("Calculator"));

         int a = 20;
         int b = 10;

         System.out.println(
               "First Number  = " + a);

         System.out.println(
               "Second Number = " + b);

         System.out.println(
               "Addition = " +
               calculator.add(a, b));

         System.out.println(
               "Subtraction = " +
               calculator.subtract(a, b));

         System.out.println(
               "Multiplication = " +
               calculator.multiply(a, b));

         System.out.println(
               "Division = " +
               calculator.divide(a, b));
      }
      catch (Exception e)
```

RSCOE-IT

```
        {
            System.out.println(
                  "Client Error: " + e
            );

            e.printStackTrace();
        }
    }
}
```

Explanation of Client Program
The client initializes the ORB:
ORB orb = ORB.init(args, null);
It then obtains the Naming Service:
orb.resolve_initial_references("NameService");
The client searches for the object named:
Calculator
using:
ncRef.resolve_str("Calculator")
The object reference is converted to the Calculator type:
CalculatorHelper.narrow(...)
The client can now invoke:
calculator.add(a, b);
Although the statement looks like a normal Java method call, the request is handled by the CORBA infrastructure and reaches the remote object.

CORBA Request Processing
When the client executes:
calculator.add(20, 10);
the following process takes place:

```
Client
  |
  | add(20,10)
  v
Client Stub
  |
  v
Client ORB
  |
  | Network Communication
  v
Server ORB
  |
  v
Server Skeleton
  |
  v
CalculatorImpl
  |
```

RSCOE-IT

```
 | 20 + 10
 v
Result = 30
 |
 v
Client
```

Execution Procedure
Step 1
Check the Java version:
java -version
and:
javac -version

Step 2
Create a working directory:
mkdir CORBAExperiment
cd CORBAExperiment

Step 3
Create the IDL file:
Calculator.idl

Step 4
Compile the IDL file using the compatible Java IDL compiler:
idlj -fall Calculator.idl

Step 5
Compile the generated Java files and application files.
For a legacy Java CORBA environment, an appropriate compilation command can be used according to the generated directory structure.
For example:
javac CalculatorApp/*.java
javac CalculatorImpl.java
javac CalculatorServer.java
javac CalculatorClient.java

Start CORBA Naming Service
In a compatible legacy Java IDL environment, start the Naming Service:
tnameserv -ORBInitialPort 1050
Expected message:
Initial Naming Context:
IOR:....
TransientNameServer:....
Keep this terminal running.

RSCOE-IT

Start the CORBA Server
Open another terminal.
Run:
java CalculatorServer -ORBInitialPort 1050
Expected output:
Calculator Server Ready...

Start the Client
Open another terminal:
java CalculatorClient -ORBInitialPort 1050
Expected output:
First Number = 20
Second Number = 10

Addition = 30
Subtraction = 10
Multiplication = 200
Division = 2.0

Sample Output
Server
Calculator Server Ready...
Client
First Number = 20
Second Number = 10

Addition = 30
Subtraction = 10
Multiplication = 200
Division = 2.0

32. Observation Table

Sr. No.	Operation	Input	Expected Output	Actual Output
1	Addition	20, 10	30	30
2	Subtraction	20, 10	10	10
3	Multiplication	20, 10	200	200
4	Division	20, 10	2.0	2.0

Running Client and Server on Different Machines
CORBA can be used in a network environment.
Example:
Machine 1 Machine 2
(Client) (Server)
| |
| |
Client Program CORBA Server
| |
v v

SCOE-IT

Client ORB <------ Network -----> Server ORB

|

v

Calculator Object

The client and server need not execute on the same computer.
The ORB configuration should identify the host and port where the Naming Service is running.
For example:

```
java CalculatorClient \
-ORBInitialHost 192.168.1.10 \
-ORBInitialPort 1050
```

The actual IP address must be replaced with the server/Naming Service machine's IP address.

Role of Naming Service
The Naming Service works like a directory.
For example:

```
Name              Object
-------------------------------
Calculator      ---> Calculator Object
```

The client asks:
Where is Calculator?
The Naming Service provides the object reference.
This avoids hard-coding the physical location of the object in the client application.

CORBA vs Local Method Invocation

Feature	Local Method	CORBA Remote Method
Object location	Same process/system	May be remote
Communication	Direct	Through ORB
Network required	No	Usually yes
Serialization	Generally not required	Required internally
Failure possibility	Lower	Network/remote failures possible
Performance	Generally faster	Network overhead
Location transparency	No	Yes

CORBA vs RMI

Feature	CORBA	Java RMI
Full Form	Common Object Request Broker Architecture	Remote Method Invocation
Interface	IDL	Java interface
Language independence	Yes	Primarily Java
Object communication	ORB	RMI infrastructure
Interoperability	High	Mainly Java ecosystem
Complexity	Higher	Comparatively simpler
Main purpose	Distributed heterogeneous objects	Distributed Java objects

---

Important CORBA Terms

Term	Meaning
CORBA	Common Object Request Broker Architecture
ORB	Object Request Broker
IDL	Interface Definition Language

RSCOE-IT

Stub	Client-side representation of remote object
Skeleton/POA	Server-side request handling mechanism
Naming Service	Locates remote objects
IIOP	Internet Inter-ORB Protocol
Remote Object	Object accessible from another process/machine
Object Reference	Reference used to access a remote object
Client	Requests service
Server	Provides service

Algorithm

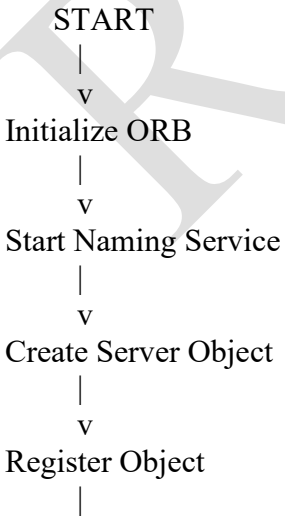
Server Algorithm

1. Start the CORBA server.
2. Initialize the ORB.
3. Create the Calculator implementation object.
4. Obtain the Naming Service reference.
5. Register the Calculator object.
6. Display server-ready message.
7. Start the ORB event loop.
8. Wait for client requests.
9. Process incoming requests.
10. Return results to clients.

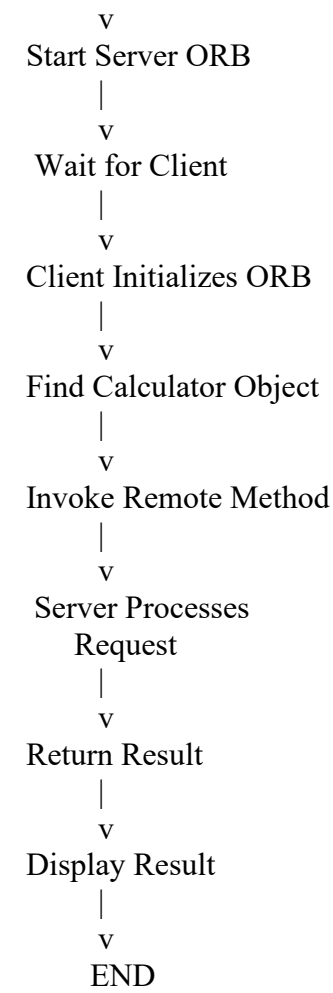
Client Algorithm

1. Start the client.
2. Initialize the ORB.
3. Obtain the Naming Service.
4. Locate the Calculator object.
5. Narrow the object reference.
6. Accept or define input values.
7. Invoke remote methods.
8. Receive results from the server.
9. Display the results.
10. Terminate the client.

Flowchart



RSCOE-IT



Result

The distributed Calculator application was successfully designed using CORBA and Java IDL. The client was able to locate the remote Calculator object through the Naming Service and invoke its methods for addition, subtraction, multiplication, and division.

Conclusion

This experiment demonstrates the fundamental principles of distributed object computing using CORBA. Students learn how:

- IDL defines a language-independent interface.
- ORB manages communication.
- Naming Service locates distributed objects.
- Client stubs and server-side components facilitate remote invocation.
- A client can invoke operations on an object located in another process or machine.

Thus, the experiment provides practical understanding of distributed object communication and middleware-based distributed computing.

KSCOE-IT

Experiment No. 8

To Use CUDA Toolkit on Windows/Linux OS to Write Programs

Aim

To study the CUDA programming model and CUDA Toolkit and develop, compile, and execute CUDA programs on Windows/Linux operating systems using an NVIDIA GPU.

Course Details

Course: Parallel & Distributed Computing
Programme: Final Year B.Tech Engineering
Experiment: CUDA Programming
Technology: NVIDIA CUDA Toolkit, CUDA C/C++
Platform: Windows / Linux
Experiment Type: GPU-Based Parallel Programming

Learning Objectives

After completing this experiment, students will be able to:

1. Explain GPU computing.
2. Understand the CUDA programming model.
3. Identify the components of the CUDA Toolkit.
4. Configure a CUDA development environment.
5. Understand CPU-GPU communication.
6. Explain CUDA threads, blocks, and grids.
7. Write CUDA kernels.
8. Allocate and transfer memory between host and device.
9. Compile CUDA programs using nvcc.
10. Execute CUDA programs on an NVIDIA GPU.
11. Develop simple parallel CUDA applications.
12. Analyze the advantages and limitations of GPU-based parallel execution.

Prerequisites

Students should have basic knowledge of:

- C/C++ programming
- Functions
- Arrays
- Pointers
- Structures
- Parallel computing
- Thread concepts
- Computer architecture
- Operating systems
- Linux/Windows command-line operations

Hardware Requirements

Minimum Recommended Hardware

- Computer/Laptop
- NVIDIA CUDA-capable GPU
- Minimum 8 GB RAM recommended
- Sufficient storage for CUDA Toolkit

RSCOE-IT

- Windows or Linux operating system

Important

CUDA programming requires an NVIDIA GPU for actual GPU execution.

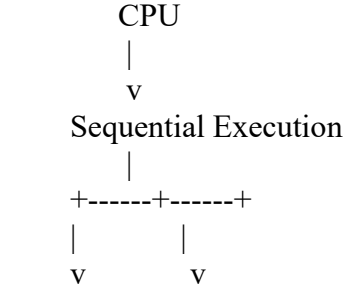
If an NVIDIA GPU is unavailable, students may study the source code and compilation concepts, but actual GPU execution requires compatible NVIDIA CUDA hardware or an appropriate CUDA-enabled remote/cloud environment.

Software Requirements

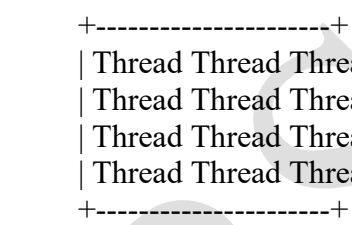
- Windows 10/11 or compatible Linux distribution
- NVIDIA GPU driver
- NVIDIA CUDA Toolkit
- C/C++ compiler
- Command Prompt/PowerShell on Windows
- Terminal on Linux
- Visual Studio integration on Windows where required

Introduction to GPU Computing

Traditional programs generally execute on the CPU.



A GPU contains a large number of processing units designed for highly parallel workloads.



Therefore, a large number of independent operations can execute concurrently.

What is CUDA?

CUDA stands for:

Compute Unified Device Architecture

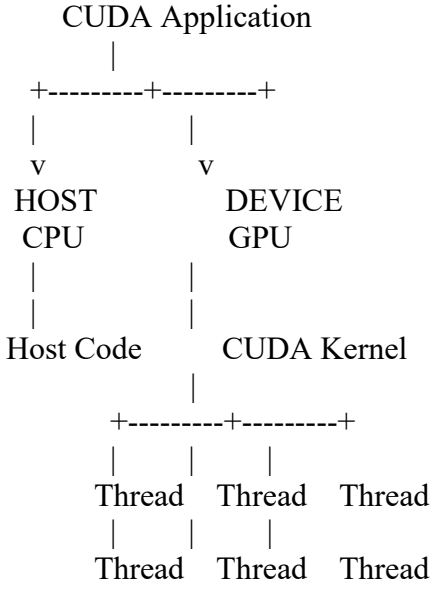
CUDA is NVIDIA's parallel computing platform and programming model that allows developers to use NVIDIA GPUs for general-purpose computation.

CUDA enables programmers to write programs in languages such as C/C++ with GPU kernels that execute on the GPU.

RSCOE-IT

CUDA Programming Model

CUDA follows a host-device programming model.



Host

The CPU is called the host.

It performs:

- Program initialization
- Memory allocation
- Data preparation
- Kernel launch
- Result retrieval

Device

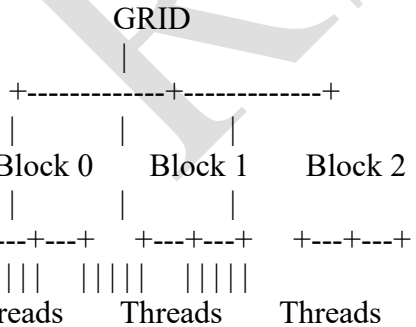
The GPU is called the device.

It executes:

- CUDA kernels
- Parallel threads
- GPU computations

CUDA Architecture

A simplified CUDA execution hierarchy is:



The hierarchy consists of:

1. Grid
2. Block

RSCOE-IT

3. Thread

CUDA Thread
A thread is the smallest execution unit in CUDA.
Example:
int idx = threadIdx.x;
This obtains the thread's index within the block.

CUDA Block
A block contains multiple threads.
For example:
Block 0

Thread 0
Thread 1
Thread 2
Thread 3
Thread 4
Thread 5
Thread 6
Thread 7
Threads within a block can cooperate using:

- Shared memory
- Synchronization
- Cooperative execution

CUDA Grid
A grid contains one or more blocks.
Grid
|
+-- Block 0
| +-- Threads
|
+-- Block 1
| +-- Threads
|
+-- Block 2
+-- Threads
A kernel launch creates a grid of blocks.

16. CUDA Kernel
A CUDA kernel is a function executed on the GPU.
A kernel is declared using:

```
__global__ void kernelFunction()
{
  // GPU code
}
```

Example:

RSCOE-IT

```
__global__ void hello()
{
   printf("Hello from GPU\n");
}
```

The kernel is launched from host code using:

```
hello<<<1, 5>>>();
```

This launches:

- 1 block
- 5 threads

CUDA Function Qualifiers

CUDA provides special function qualifiers.

Qualifier	Meaning
__global__	Function executes on GPU and is called from CPU
__device__	Function executes on GPU and is called from GPU
__host__	Function executes on CPU
__host__ __device__	Function can execute on CPU and GPU

CUDA Memory Model

CUDA provides different types of memory.

Memory	Location	Scope	Typical Characteristic
Registers	GPU	Thread	Very fast
Local Memory	GPU	Thread	Private to thread
Shared Memory	GPU	Block	Fast and shared by block threads
Global Memory	GPU	Grid/application	Large but relatively slower
Constant Memory	GPU	Grid/application	Read-only
Texture/Read-only Memory	GPU	Grid/application	Specialized read access

Basic model:

```
              GPU
               |
     +----------+----------+
     |         |          |
  Registers   Shared     Global
    |        Memory      Memory
  Thread      Block       Grid
```

CUDA Execution Flow

A typical CUDA program follows these steps:

```
Start
 |
 v
Initialize CPU program
 |
 v
Allocate Host Memory
 |
 v
Allocate Device Memory
 |
```

PSCOE-IT

v
Copy Data: Host -> Device
|
v
Launch CUDA Kernel
|
v
GPU Parallel Execution
|
v
Copy Result: Device -> Host
|
v
Free Device Memory
|
v
Display Result
|
v
End

CUDA Toolkit
The CUDA Toolkit provides the tools and libraries required to develop CUDA applications.
Important components include:

- CUDA compiler nvcc
- CUDA runtime libraries
- CUDA libraries
- Development tools
- GPU debugging/profiling tools
- CUDA headers
- Samples and documentation

Check CUDA Installation
Open a terminal or command prompt.
Check the CUDA compiler:
nvcc --version
or:
nvcc -V
A valid CUDA installation should display CUDA compiler information.

Check NVIDIA GPU
On Linux and many NVIDIA driver installations, use:
nvidia-smi
The command displays information such as:

- GPU model
- Driver version
- CUDA compatibility information
- GPU memory
- GPU utilization

RSCOE-IT

On Windows, nvidia-smi can also be executed from Command Prompt or PowerShell when the NVIDIA driver installation provides it.

Experiment 1 – Hello CUDA Program
Aim
To write a simple CUDA program that executes a kernel on the GPU.

Program
Create:
hello.cu
Program:

```
#include <stdio.h>
#include <cuda_runtime.h>

__global__ void helloCUDA()
{
   printf("Hello from CUDA Thread %d\n",
         threadIdx.x);
}

int main()
{
   printf("Hello from CPU\n");

   helloCUDA<<<1, 5>>>();

   cudaDeviceSynchronize();

   return 0;
}
```

Explanation
Kernel
__global__ void helloCUDA()
Declares a GPU kernel.
Thread ID
threadIdx.x
Returns the thread's index inside the block.
Kernel Launch
helloCUDA<<<1, 5>>>();
The execution configuration:
<<<number_of_blocks, threads_per_block>>>
Here:
Blocks = 1
Threads = 5
Therefore, five GPU threads execute the kernel.
Synchronization
cudaDeviceSynchronize();

RSCOE-IT

Waits for GPU operations to complete before the CPU continues.

Compile and Execute on Linux
Save the file as:
hello.cu
Compile:
nvcc hello.cu -o hello
Execute:
./hello
Expected output:
Hello from CPU
Hello from CUDA Thread 0
Hello from CUDA Thread 1
Hello from CUDA Thread 2
Hello from CUDA Thread 3
Hello from CUDA Thread 4
The order of thread-generated output should not be assumed in more complex concurrent programs.

Compile and Execute on Windows
Open Command Prompt, PowerShell, or the appropriate CUDA-enabled development environment.
Navigate to the program directory:
cd C:\CUDAPrograms
Compile:
nvcc hello.cu -o hello.exe
Run:
.\hello.exe
Expected output:
Hello from CPU
Hello from CUDA Thread 0
Hello from CUDA Thread 1
Hello from CUDA Thread 2
Hello from CUDA Thread 3
Hello from CUDA Thread 4

Experiment 2 – CUDA Vector Addition
Aim
To implement vector addition using CUDA threads.

Problem Statement
Given two arrays:
A = [10, 20, 30, 40, 50]
B = [1, 2, 3, 4, 5]
Calculate:
C = A + B
Expected result:
C = [11, 22, 33, 44, 55]

RSCOE-IT

Algorithm

1. Initialize two input arrays.
2. Allocate memory on the CPU.
3. Allocate memory on the GPU.
4. Copy input arrays from host to device.
5. Launch the CUDA vector addition kernel.
6. Calculate the global thread index.
7. Add corresponding elements.
8. Copy the result from device to host.
9. Display the result.
10. Free GPU memory.

CUDA Program – Vector Addition

Create:

vector_add.cu

```
#include <stdio.h>
#include <cuda_runtime.h>

__global__ void vectorAdd(int *A, int *B, int *C, int N)
{
   int i = blockIdx.x * blockDim.x + threadIdx.x;

   if(i < N)
   {
      C[i] = A[i] + B[i];
   }
}

int main()
{
   const int N = 5;

   int h_A[N] = {10, 20, 30, 40, 50};
   int h_B[N] = {1, 2, 3, 4, 5};
   int h_C[N];

   int *d_A, *d_B, *d_C;

   cudaMalloc((void**)&d_A, N * sizeof(int));
   cudaMalloc((void**)&d_B, N * sizeof(int));
   cudaMalloc((void**)&d_C, N * sizeof(int));

   cudaMemcpy(d_A, h_A,
            N * sizeof(int),
            cudaMemcpyHostToDevice);
```

RSCOE-IT

```
    cudaMemcpy(d_B, h_B,
            N * sizeof(int),
            cudaMemcpyHostToDevice);

    vectorAdd<<<1, N>>>(d_A, d_B, d_C, N);

    cudaDeviceSynchronize();

    cudaMemcpy(h_C, d_C,
            N * sizeof(int),
            cudaMemcpyDeviceToHost);

    printf("Vector Addition Result:\n");

    for(int i = 0; i < N; i++)
    {
       printf("%d + %d = %d\n",
            h_A[i], h_B[i], h_C[i]);
    }

    cudaFree(d_A);
    cudaFree(d_B);
    cudaFree(d_C);

    return 0;
}
```

Explanation of Vector Addition Program
Global Thread Index
int i = blockIdx.x * blockDim.x + threadIdx.x;
This calculates the unique global thread index.
Where:

- blockIdx.x = block index
- blockDim.x = number of threads per block
- threadIdx.x = thread index within the block

Memory Allocation
cudaMalloc(...)
Allocates memory on the GPU.

Host to Device Transfer
cudaMemcpy(...,
cudaMemcpyHostToDevice);
Copies data from CPU memory to GPU memory.

Kernel Execution
vectorAdd<<<1, N>>>(d_A, d_B, d_C, N);
Launches the vector addition kernel.

RSCOE-IT

Device to Host Transfer
cudaMemcpy(...,
cudaMemcpyDeviceToHost);
Copies results from GPU memory back to CPU memory.

Expected Output
Vector Addition Result:

10 + 1 = 11
20 + 2 = 22
30 + 3 = 33
40 + 4 = 44
50 + 5 = 55

Experiment 3 – CUDA Parallel Array Addition
For a larger array, multiple blocks can be used.
Kernel

```
__global__ void vectorAdd(int *A, int *B, int *C, int N)
{
   int index =
      blockIdx.x * blockDim.x + threadIdx.x;

   if(index < N)
   {
      C[index] = A[index] + B[index];
   }
}
```

For example:

```
int threads = 256;

int blocks =
   (N + threads - 1) / threads;

vectorAdd<<<blocks, threads>>>(
   d_A, d_B, d_C, N);
```

This allows the program to process a large number of elements using multiple blocks.

Why Use This Formula?
(N + threads - 1) / threads
calculates the number of blocks required to process N elements when each block contains threads threads.
Example:
N = 1000
Threads per block = 256

Blocks = (1000 + 256 - 1) / 256
= 1255 / 256
= 4
Therefore:

RSCOE-IT

4 blocks × 256 threads
provides 1024 thread positions, enough to process 1000 elements.
The condition:
if(index < N)
prevents extra threads from accessing invalid array elements.

Important CUDA Built-in Variables

Variable	Meaning
threadIdx.x	Thread index
threadIdx.y	Thread index in Y dimension
threadIdx.z	Thread index in Z dimension
blockIdx.x	Block index
blockDim.x	Number of threads in a block
gridDim.x	Number of blocks in grid

CUDA Execution Configuration
CUDA kernel launches commonly use:
kernel<<<blocks, threads>>>();
For example:
vectorAdd<<<4, 256>>>();
means:
Grid
|
+-- Block 0 → 256 threads
+-- Block 1 → 256 threads
+-- Block 2 → 256 threads
+-- Block 3 → 256 threads
Total thread positions:
4 × 256 = 1024

CPU vs GPU Execution

Feature	CPU	GPU
Main purpose	General computation	Highly parallel computation
Processing cores	Relatively fewer	Many parallel processing resources
Parallelism	Moderate	Very high for suitable workloads
Memory hierarchy	CPU caches/RAM	GPU memory hierarchy
Best suited for	Branch-heavy/general tasks	Data-parallel tasks
Programming	C/C++/Java/Python etc.	CUDA C/C++ and related APIs
Example	Sequential loop	CUDA kernel

CUDA Memory Transfer
A major operation in CUDA programming is data transfer.
CPU Memory
|
| cudaMemcpy()
| Host -> Device
v
GPU Memory
|

RCOE-IT

```
  | Kernel execution
  v
GPU Result
  |
  | cudaMemcpy()
  | Device -> Host
  v
CPU Memory
```

Important CUDA Runtime Functions

Function	Purpose
cudaMalloc()	Allocate GPU memory
cudaFree()	Free GPU memory
cudaMemcpy()	Copy data
cudaDeviceSynchronize()	Wait for device operations
cudaGetDeviceCount()	Get number of CUDA devices
cudaGetDeviceProperties()	Get GPU information
cudaMemset()	Initialize device memory

CUDA Error Handling
CUDA programs should check errors.
Example:

```
cudaError_t error;

error = cudaGetLastError();

if(error != cudaSuccess)
{
   printf("CUDA Error: %s\n",
        cudaGetErrorString(error));
}
```

After a kernel launch:

```
kernel<<<blocks, threads>>>();

cudaError_t error = cudaGetLastError();

if(error != cudaSuccess)
{
   printf("Kernel Error: %s\n",
        cudaGetErrorString(error));
}
```

Measuring CUDA Execution Time
CUDA provides CUDA events for timing GPU operations.
Example:

```
cudaEvent_t start, stop;

cudaEventCreate(&start);
cudaEventCreate(&stop);
```

RSCOE-IT

```
cudaEventRecord(start);

kernel<<<blocks, threads>>>();

cudaEventRecord(stop);

cudaEventSynchronize(stop);

float milliseconds = 0;

cudaEventElapsedTime(
   &milliseconds,
   start,
   stop);

printf("GPU Time = %f ms\n",
      milliseconds);
```

This can be used to study the performance of GPU kernels.

Important CUDA Concepts
Students should understand:
1. Host
CPU-side execution environment.
2. Device
GPU-side execution environment.
3. Kernel
Function executed by GPU threads.
4. Thread
Smallest execution unit.
5. Block
Group of cooperating threads.
6. Grid
Collection of blocks executing a kernel.
7. Global Memory
Large GPU memory accessible by threads.
8. Shared Memory
Fast memory shared by threads in a block.

Applications of CUDA
CUDA is widely applicable to workloads such as:

- Scientific computing
- Image processing
- Video processing
- Machine learning
- Deep learning
- Matrix operations
- Computer vision
- Simulation

RSCOE-IT

- Financial computation
- Data analytics
- Signal processing
- Molecular modeling
- Numerical computation

Advantages of CUDA

1. Provides massive parallelism for suitable workloads.
2. Can accelerate data-parallel computations.
3. Provides a programming model based on C/C++.
4. Supports many CUDA libraries.
5. Provides different levels of GPU memory.
6. Supports thread-level parallelism.
7. Provides GPU profiling and debugging tools.
8. Suitable for large computational workloads.

Limitations of CUDA

1. CUDA is primarily associated with NVIDIA GPU hardware.
2. GPU memory is separate from normal CPU memory.
3. Host-device data transfers can introduce overhead.
4. Not every algorithm benefits from GPU execution.
5. Thread divergence can reduce efficiency.
6. Memory access patterns affect performance.
7. GPU programming requires understanding parallel execution.
8. Hardware and software compatibility must be considered.

Important CUDA Performance Considerations

For better performance, students should consider:

- Minimize unnecessary host-device transfers.
- Use suitable block sizes.
- Access global memory efficiently.
- Use shared memory where appropriate.
- Avoid unnecessary thread divergence.
- Maximize useful parallelism.
- Measure execution time rather than assuming GPU execution is always faster.
- Use CUDA profiling tools for optimization.

Suggested Matrix Addition Kernel

```
__global__ void matrixAdd(
   int *A,
   int *B,
   int *C,
   int N)
{
   int row =
      blockIdx.y * blockDim.y + threadIdx.y;
```

RSCOE-IT

```
    int col =
        blockIdx.x * blockDim.x + threadIdx.x;

    if(row < N && col < N)
    {
        int index = row * N + col;

        C[index] =
            A[index] + B[index];
    }
}
```

Example execution configuration:

```
dim3 threads(16,16);

dim3 blocks(
    (N + 15) / 16,
    (N + 15) / 16
);

matrixAdd<<<blocks, threads>>>(
    d_A, d_B, d_C, N);
```

This demonstrates two-dimensional CUDA thread organization.

Observation Table

Sr. No.	Program	Input	Expected Result	Actual Result
1	Hello CUDA	5 threads	5 thread messages	
2	Vector Addition	A, B	A+B	
3	Vector Subtraction	A, B	A-B	
4	Vector Multiplication	A, B	A×B	
5	Array Maximum	Array	Maximum value	
6	Matrix Addition	A, B	A+B	
7	Matrix Multiplication	A, B	A×B	

Troubleshooting

Problem	Possible Reason	Solution
nvcc not found	CUDA Toolkit not in PATH/not installed	Verify installation and environment configuration
no CUDA-capable device	NVIDIA GPU unavailable/inaccessible	Verify GPU and driver
Compilation error	CUDA/compiler mismatch	Check supported CUDA/JDK/compiler environment as applicable
Kernel gives wrong result	Incorrect thread indexing	Verify grid/block/thread calculations
Illegal memory access	Invalid array index	Add bounds checking
Program is slow	Excessive memory transfer	Reduce unnecessary transfers
GPU program crashes	Invalid memory access or kernel error	Add CUDA error checking
nvidia-smi	Driver/tool not installed or PATH	Verify NVIDIA driver installation

RSCOE-IT

unavailable	issue	

Result

The CUDA Toolkit environment was studied and configured on Windows/Linux. CUDA programs were developed using CUDA C/C++, compiled using nvcc, and executed using an NVIDIA GPU. The experiment demonstrated the CUDA programming model, including host-device execution, kernels, threads, blocks, grids, GPU memory allocation, and data transfer.

Conclusion

This experiment provides practical understanding of GPU-based parallel programming using NVIDIA CUDA.

Students learn how a conventional sequential problem can be transformed into a parallel CUDA application by:

Sequential Problem
|
v
Identify Parallel Work
|
v
Create CUDA Kernel
|
v
Organize Threads
|
v
Configure Blocks/Grid
|
v
Execute on GPU
|
v
Collect Results

CUDA provides a practical framework for developing high-performance parallel applications for suitable data-parallel workloads.

RSCOE-IT